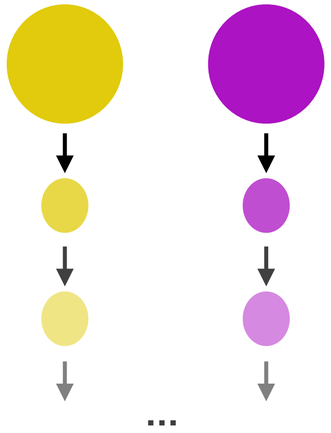
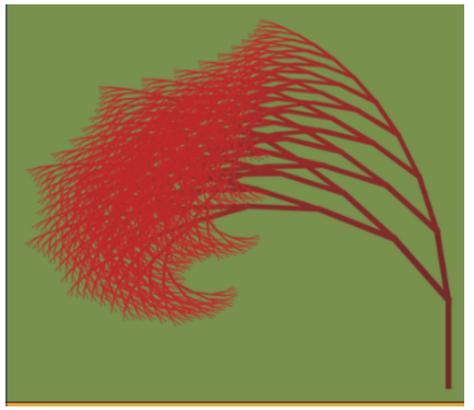


$\tau \quad \tau \quad \tau \ldots \tau \quad \tau \quad \tau$



Enhanced Coinduction and Interaction Trees

Refactoring a proof library, with theory and history

Roger Burtonpatel, 6.26

ITrees Refactor

Experiment Report

The talk from a 10,000 meter view:

I ported the ITrees library, which uses coinduction,
from the **paco** library to the **rocq-coinduction** library.

In this talk, I explain:
1. some theoretical foundations of coinduction
2. The evolution of those techniques in rocq
3. Tradeoffs and improvements of the experiment

ITrees Refactor: the tl;dr

“I refactored the ITrees library”

60% Typing

40% Thinking, Learning, (Re) Designing

I assume familiarity with proof assistants

Ricky the Rocq says:

“here’s how I would think about it a proof assistant (specifically Rocq).”

Don’t hesitate to ask about unfamiliar notation or concepts!

Intro: Equality

When does `x = x` ?


`reflexivity`

When does `x = y` ?


`exists P : x = y`

When does `(f : A -> B) = g` ?


`Axiom` needed (extensionality)

That is, if we want `f` and `g` to be the same *mathematical function…*

Reason we need Extensional Equality:

You can’t just look at functions syntactically.

`induction f` doesn’t work…

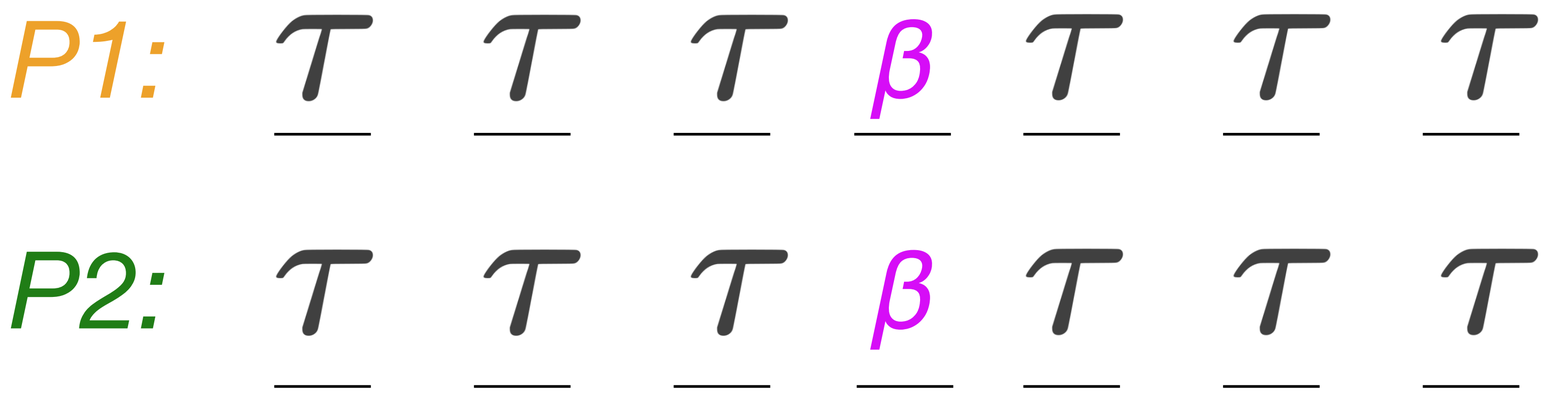
But… Extensionality does work!*

```
Axiom functional_extensionality :

    forall x, f x = g x -> f = g.
```

* it is a sound axiom w.r.t. the rest of Rocq

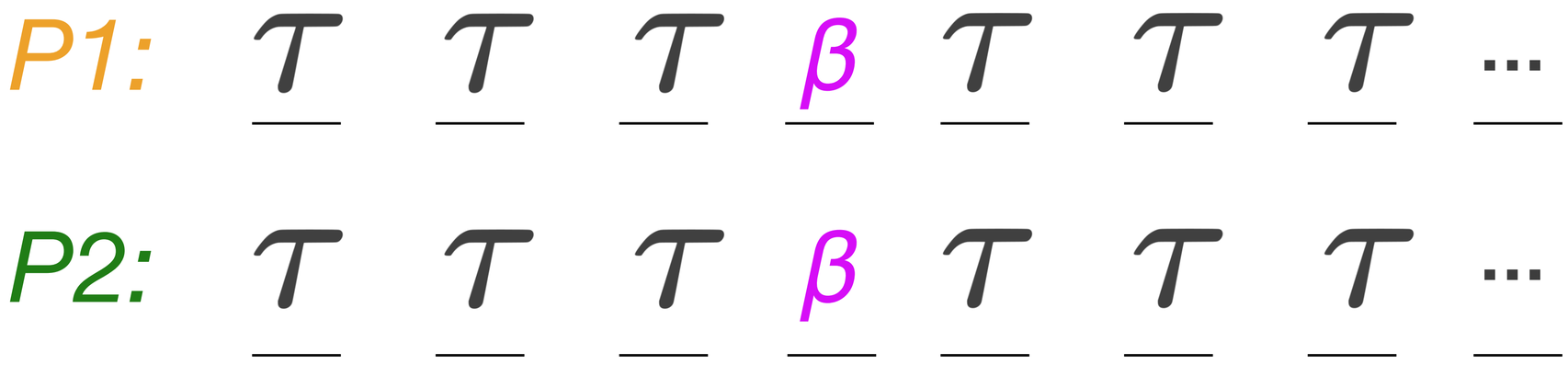
Extensionality Extended: Processes



Idea 1: If all the observations τ or β within the process align,

the two processes are extensionally equal

Extensionality Extended: Processes

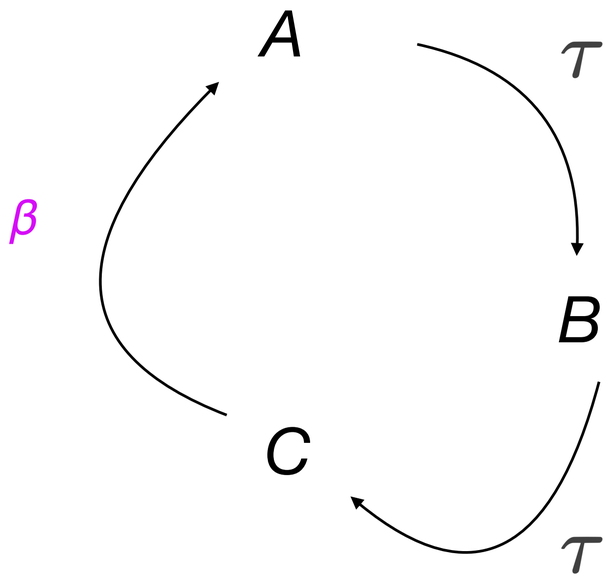


Idea 1: If all the observations τ or β within the process align, the two processes are extensionally equal

Idea 2: This can work even when the processes are *infinite*

Extensionality Generalized: Non-Wf Objects

Two Prominent Examples:



$\tau \quad \tau \quad \tau \quad \beta \quad \tau \quad \tau \quad \tau \quad \ldots$

Label Transition Systems (LTSs)

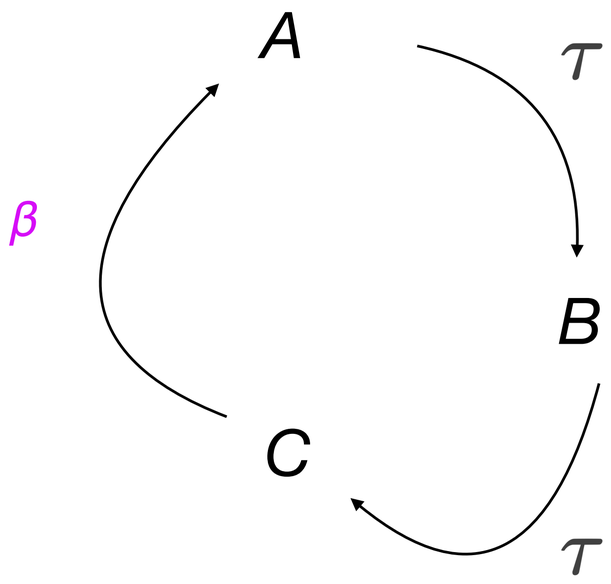
(Often) Defined with `Prop`

Infinite Objects (Processes, Streams…)

Defined with `CoFixpoint`

Extensionality Generalized: Non-Wf Objects

Two Prominent Examples:



$\tau \quad \tau \quad \tau \quad \beta \quad \tau \quad \tau \quad \tau \quad \ldots$

```
CoInductive stream :=
 | τ (s : stream)
 | β (n : nat) (s : stream).
```

```
CoFixpoint forever := τ τ τ (β 1) τ τ τ forever.
```

Label Transition Systems (LTSs)

(Often) Defined with `Prop`

Infinite Objects (Processes, Streams…)

Defined with `CoFixpoint`

Focus of this talk

What kind of *Equality* is this?

Not syntactic… `x = x`

(**`reflexivity`**)

Rather *observational* (aka *bisimilarity)*

`x ≈ y`

Pseudo-

`observe x = observe y` **(forever)**

But we don’t want to probe inputs forever…

The need for finite infinite proof

But we don’t want to probe inputs forever…

With circular or infinite proofs, there are pitfalls.

Case study:

Idiot robot (Claude) is not always clever enough to avoid them!

The need for finite infinite proof

But we don’t want to probe inputs forever…

With circular or infinite proofs, there are pitfalls.

Case study:

Idiot robot (Claude) is not always clever enough to avoid them!

Prompt: Prove weak bisimilarity up-to ≳

(one-sided weak-strong bisimilarity):

```
forall (c : Chain eutt),

Proper (euttge (E := E) eq ==> euttge eq ==> flip impl) (elem c _ _ RR).
```

The need for finite infinite proof

But we don’t want to probe inputs forever…

With circular or infinite proofs, there are pitfalls.

Case study:

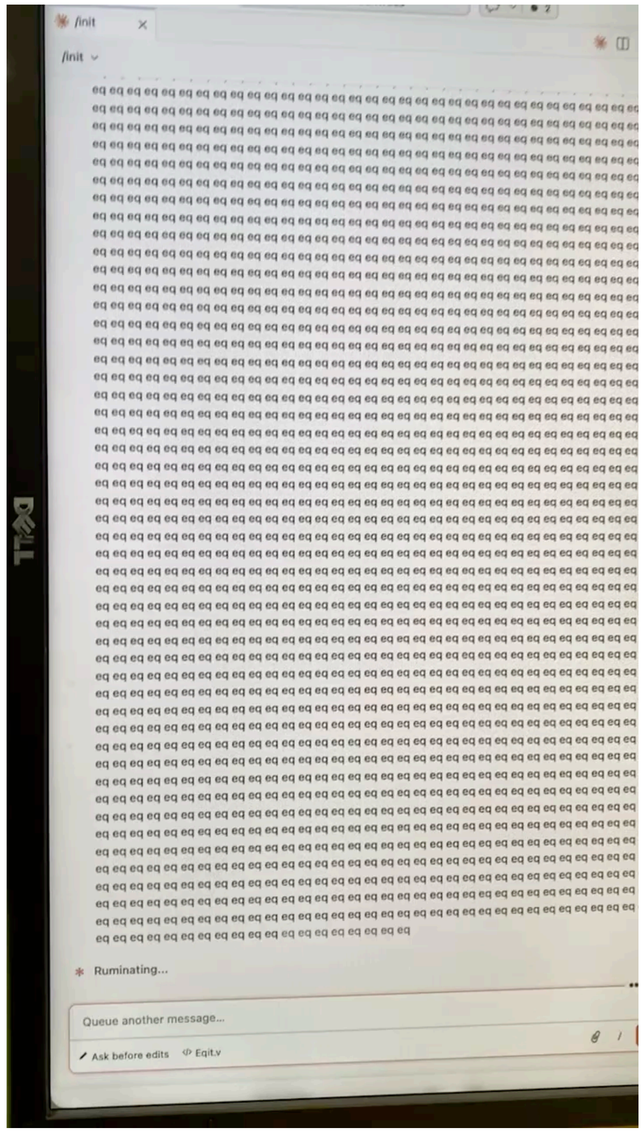
Idiot robot (Claude) is not always clever enough to avoid them!

Prompt: Prove weak bisimilarity up-to ≳

(one-sided weak-strong bisimilarity):

```
forall (c : Chain eutt),

Proper (euttge (E := E) eq ==> euttge eq ==> flip impl) (elem c _ _ RR).
```



We want to keep our potentially-looping objects in sync.
This is what Coinduction is for.

The Slide to Pay Attention To™

“Coinduction is needed for writing an infinite proof”

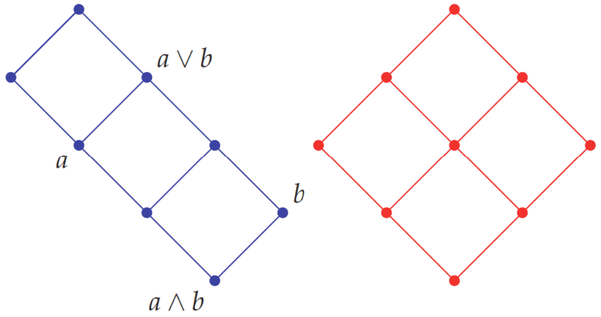
Some Background

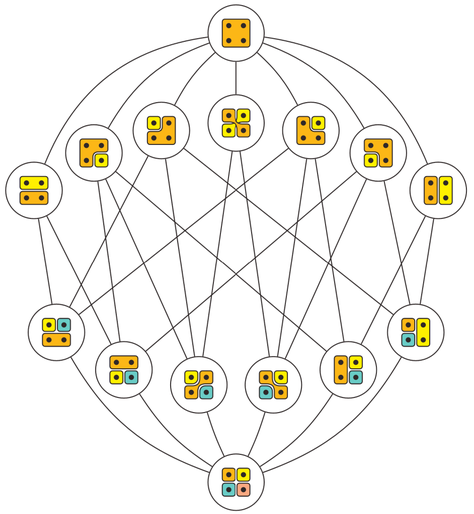
Coinduction in general admits models in terms of final coalgebras

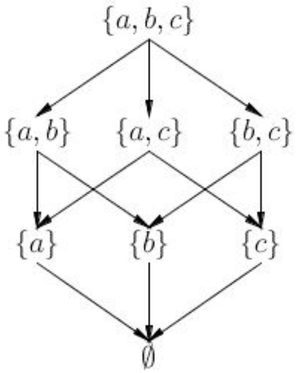
But also order- and lattice-theoretic ones in terms of greatest fix points

This latter is simpler and sufficient for defining relations,
and is the focus of this talk

Review: Lattices





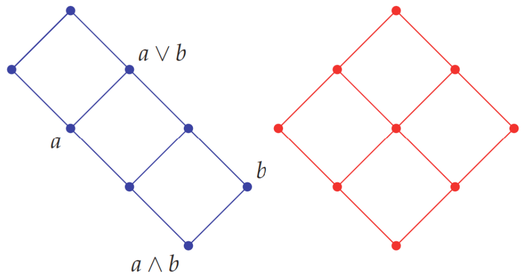


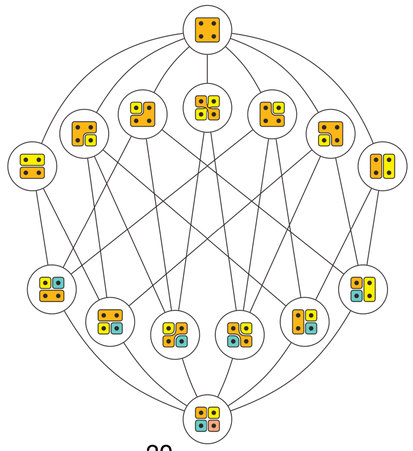
Is a superset of $\supseteq$

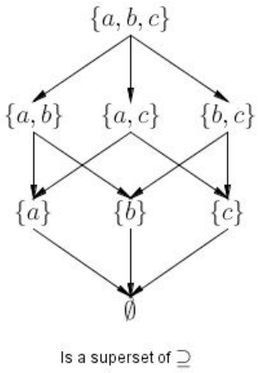
Review: Lattices

- A lattice is a set, a preorder defined on elements of the set, and supremum (least upper bound) and infimum (greatest lower bound) operators.
- From now on, we will write this as $\langle X, \leqslant, \bigvee \rangle$

(greatest lower bounds can be derived from least upper bounds) w.l.o.g.





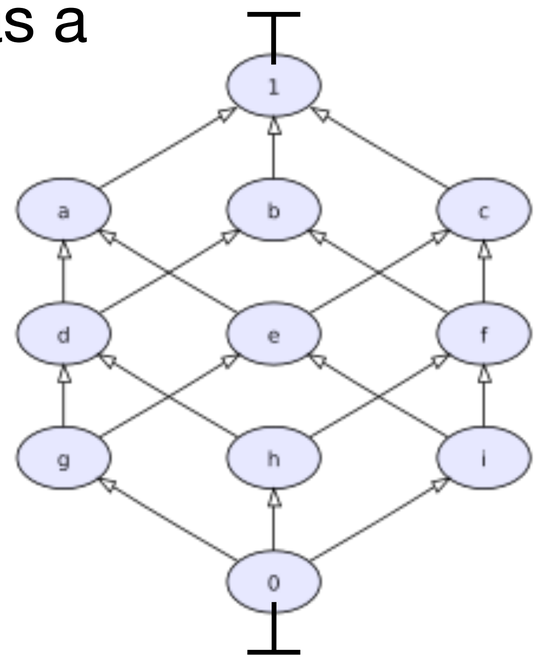


Review: Lattices

- A *complete* lattice is a lattice in which **any subset of *X* admits a least upper bound and greatest lower bound.**

- A natural consequence of this is that every complete lattice has a maximum element (top, $\top$) and minimum element (bottom, $\bot$)

$$\langle X, \leqslant, \bigvee \rangle + \top \bot$$



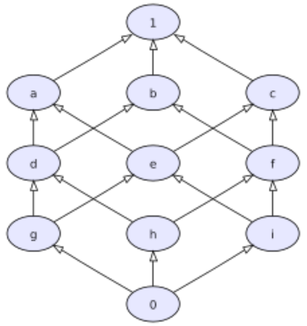
Review: Lattices

- A function on a lattice is *monotone* if it respects the preorder.

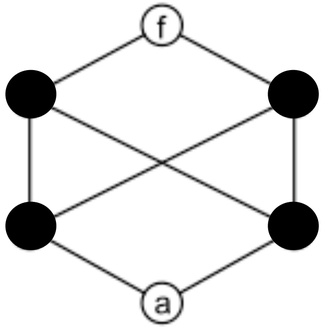
That is, *mon f* means $\forall\ x\ y,\ x \leqslant y \rightarrow f\ x \leqslant f\ y$.

- A *fix point* of a function *f* is an element *x* s.t. $f\ x = x$.

- **Every monotone function on a complete lattice admits a complete lattice of its fix points (Knaster-Tarski)**

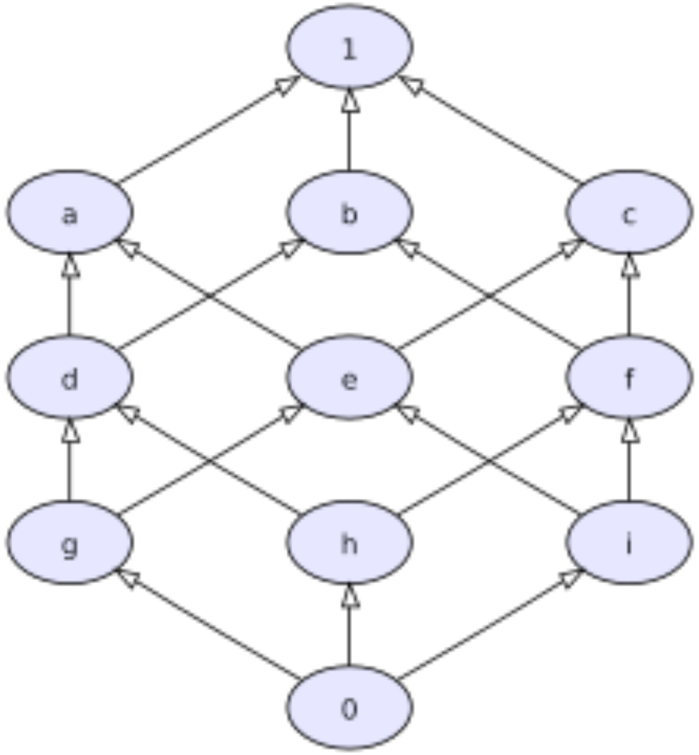


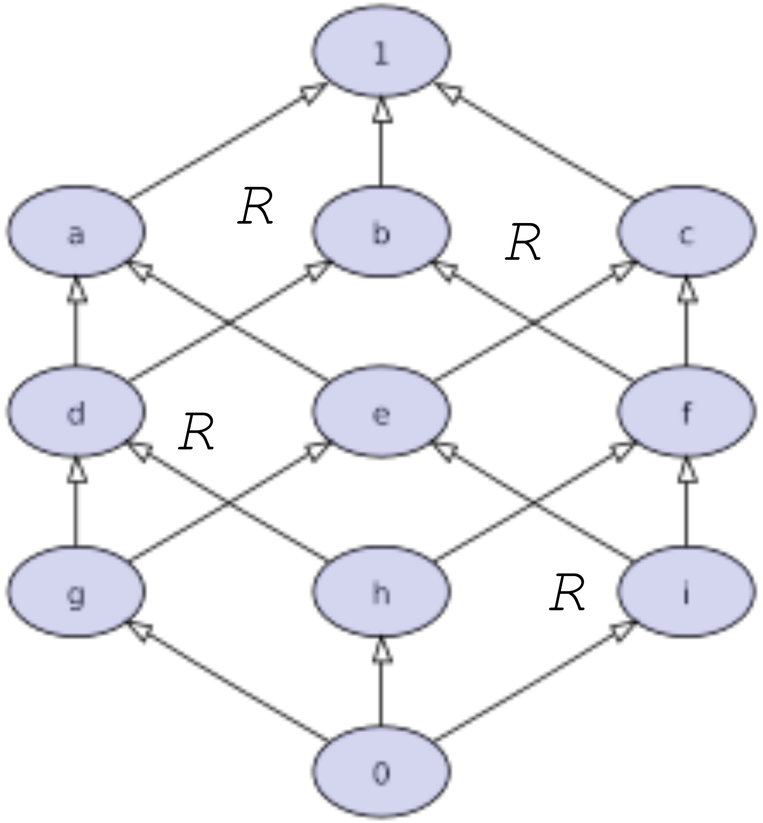
$\rightarrow b \rightarrow$



This means every monotone function has a *greatest fix point* - this will be important!

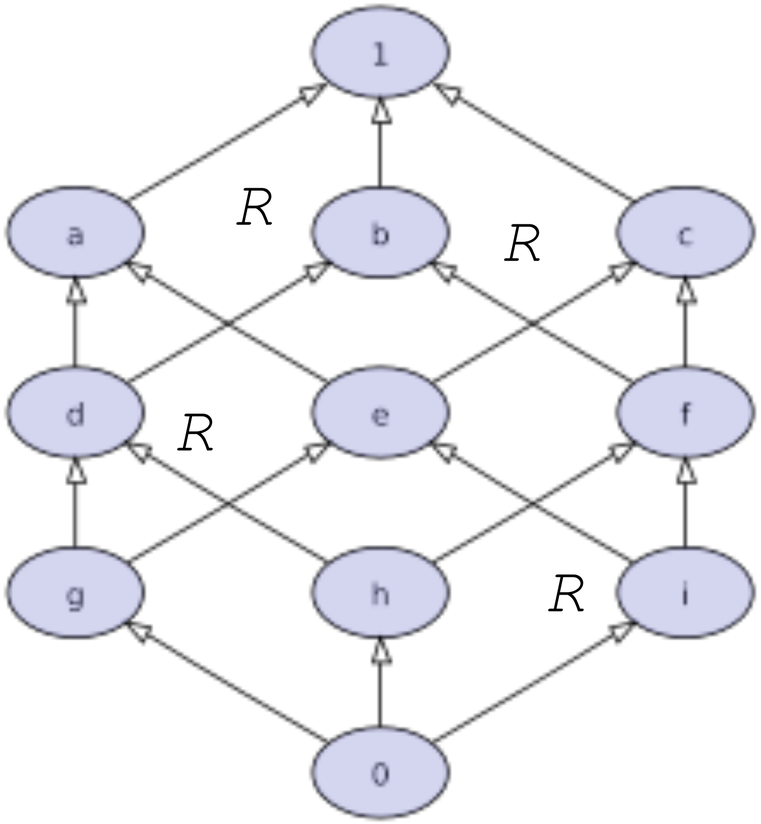
We can have Lattices of Elements, of Relations, and so on…





…

Focus: Lattices of Relations



These lattices exist because:

1. `Prop` is a complete lattice, with `<= ≜ ->`
2. If `X` is a complete lattice, then `A -> X` is a complete lattice

`<=` on lattices of relations is determined *pointwise:*

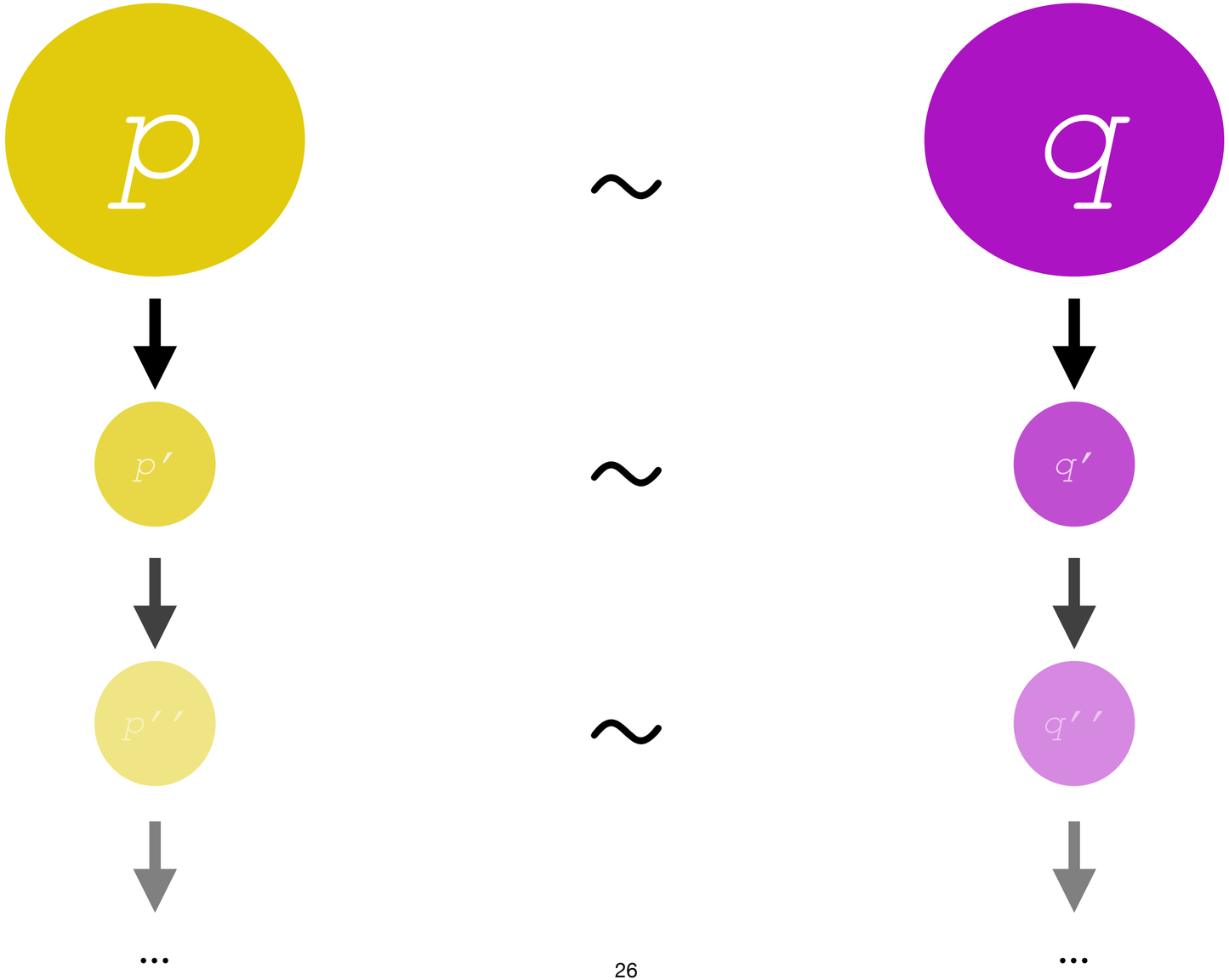
`R <= S` when `forall x, R x <= S x`

Our study: lattice of relations on states:

```
state -> state -> Prop
```

Why is this important for proving observational equivalence?

We need to be able to show objects “observe” the same *forever*.



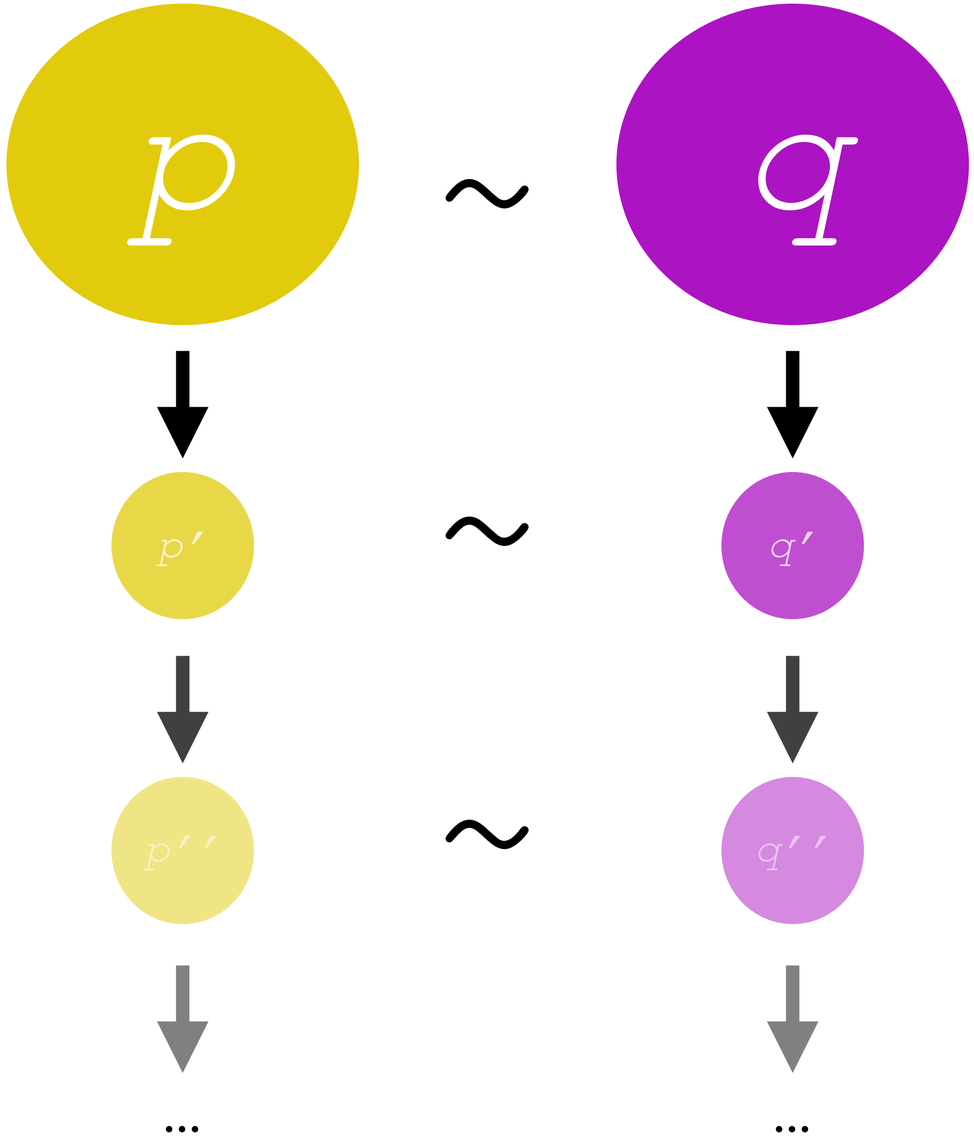
Case study: Coinduction for Bisimulation

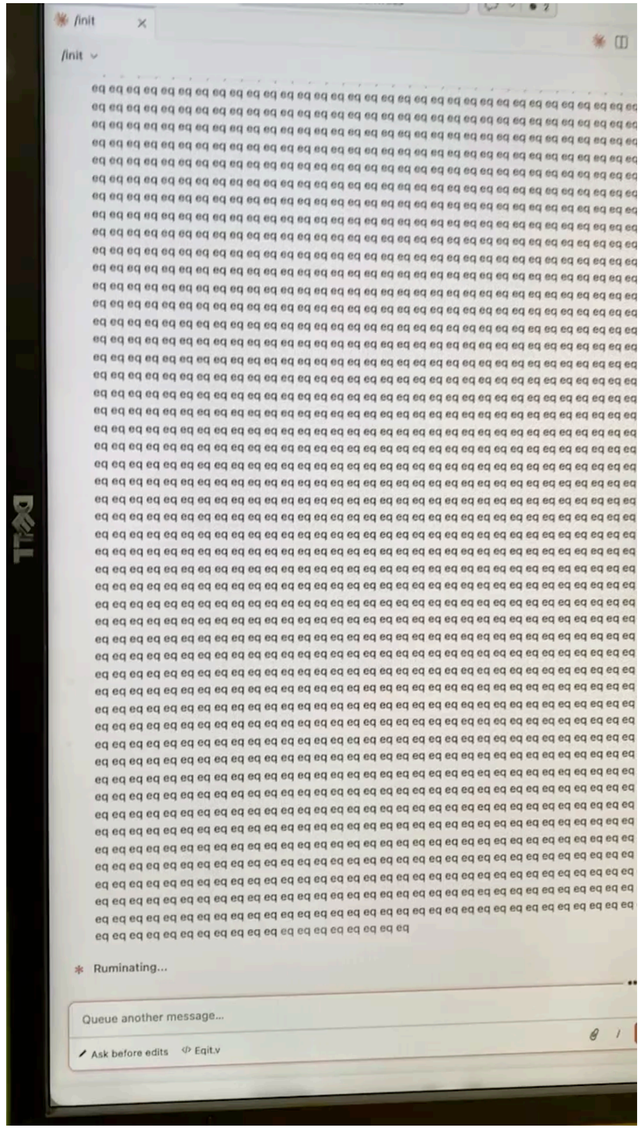
Core case study, an instance of this framework

$$\underline{\tau \quad \tau \quad \tau \quad \beta \quad \tau \quad \tau \quad \tau \quad \ldots}$$

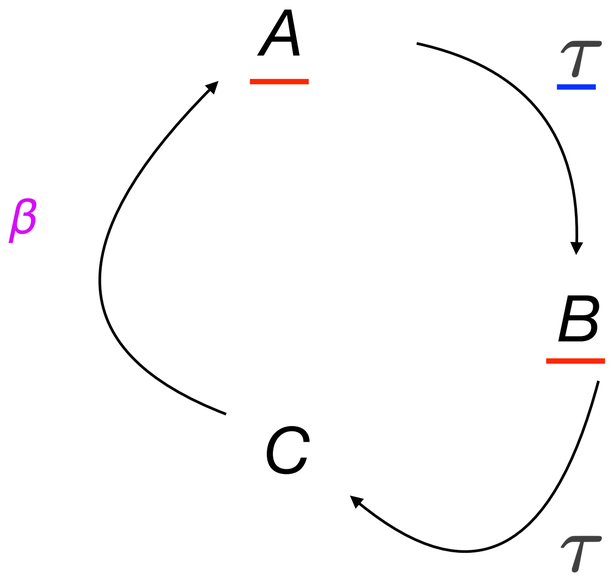
$$\tau \quad \tau \quad \tau \quad \beta \quad \tau \quad \tau \quad \tau \quad \ldots$$

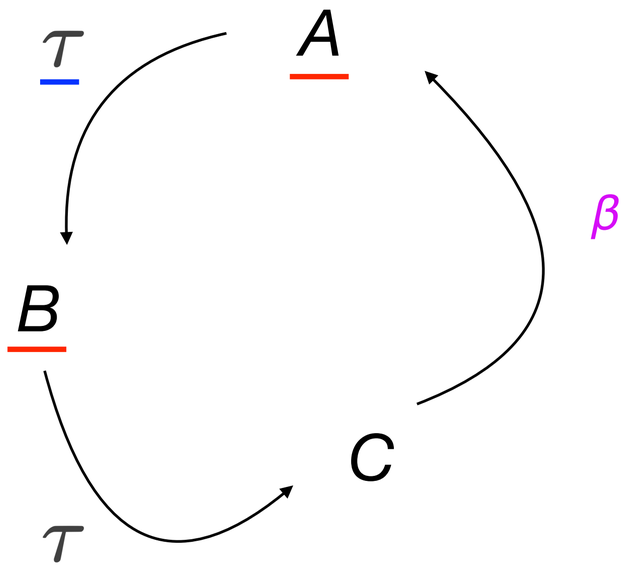
But we can’t do this manually…





Instance: Observational equivalence for LTSs





Need a relation that:

1. Shows steps are synchronized
2. Is preserved by a step

The way forward: A lattice!

Recall: Fix points

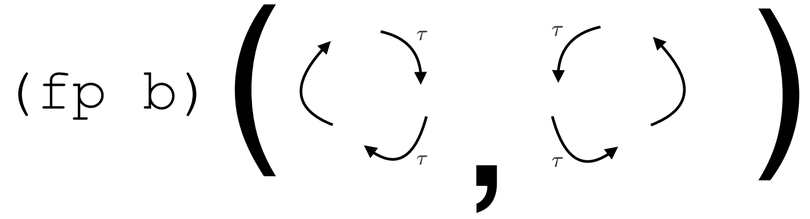
`x = b x` : `x` is a **fix point** of `b`

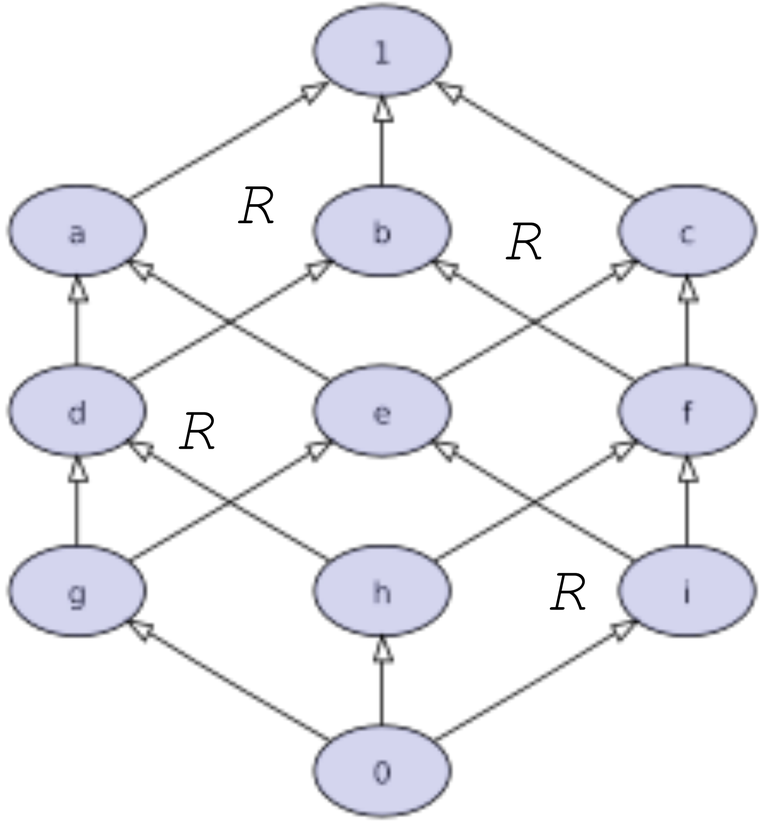
If we could find a fix point that *is itself a relation*,

`(fp b) = b (fp b)` : `(fp b)` is a **fix point** of `b`

and relate our systems by `(fp b)`,

we could show stability under `b` for infinite steps.

`(fp b)` 



Fortunately… we know `(fp b)` exists!

(for any monotone `b`)

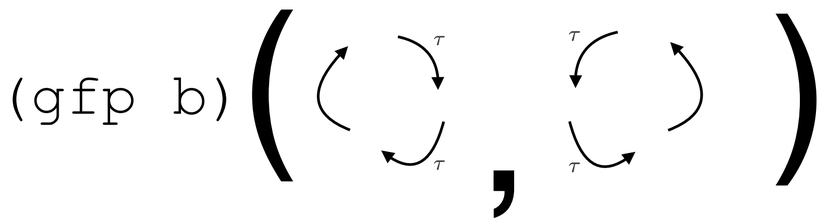
Recall Knaster-Tarski:

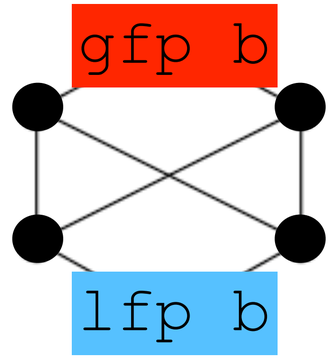
Every monotone function on a complete lattice admits a complete lattice of its fix points.

We know (at least two) such relations exists:

The **least** and **greatest** fix points of any monotone function on our system!

Our systems may run forever, so we pick the **greatest** fix point.

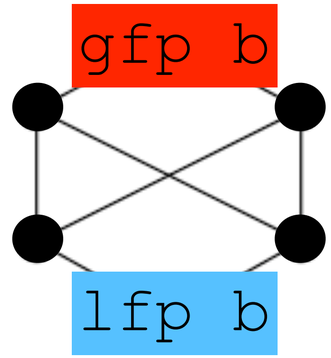




Two questions remain:

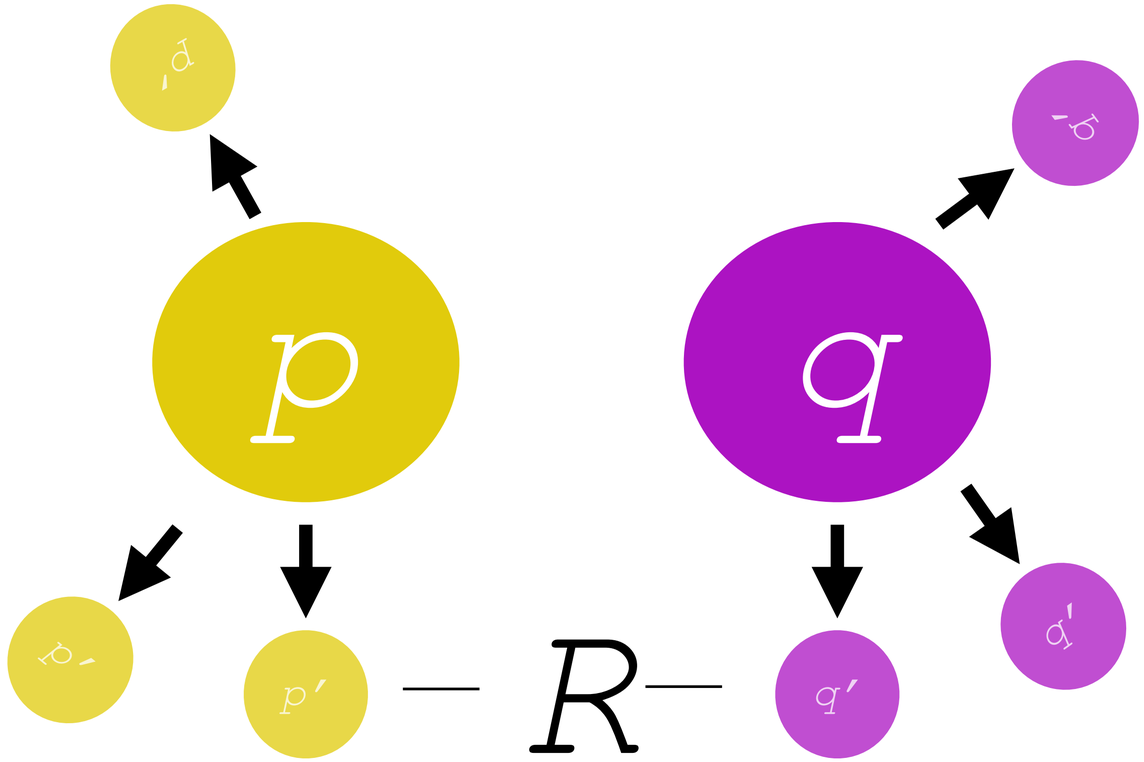
1. How do we pick `b`?

2. How do we make its fix point be a relation, and how do we know that the lattice it lives in has the properties we want?



Defining b: preservation by step

b(R, p, q) when

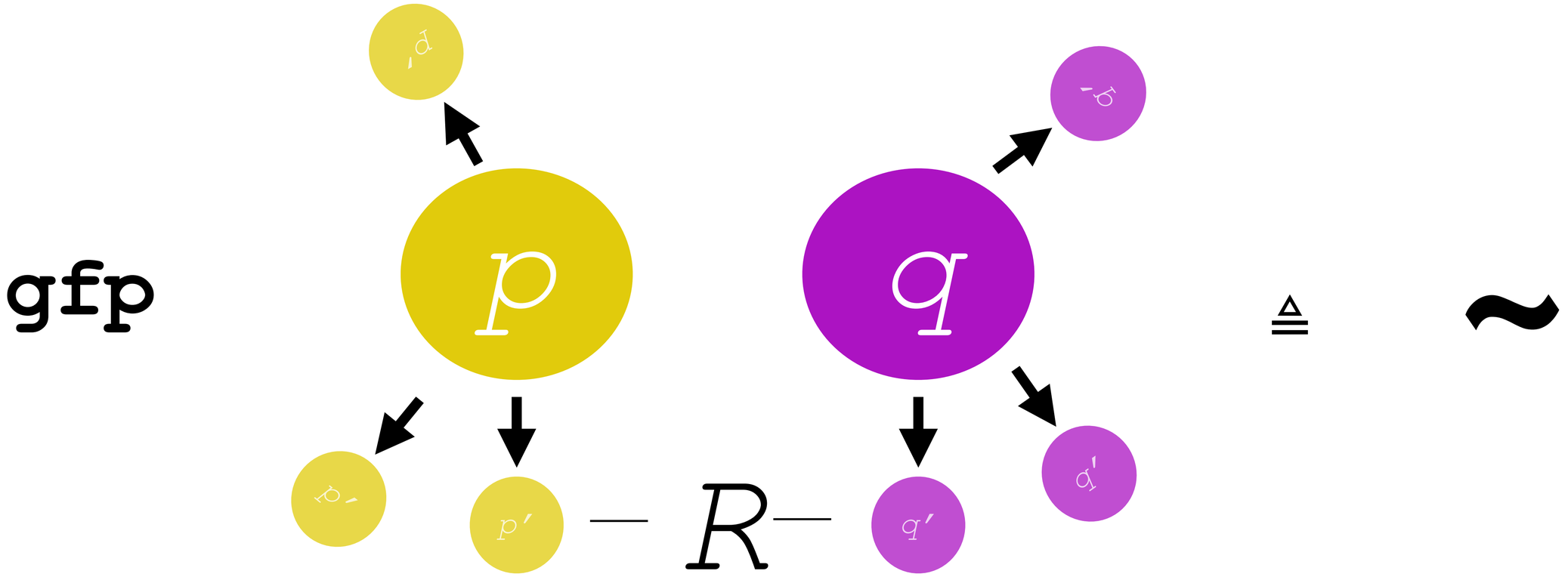


"p and q take the same step together and end back in R"

Formally,

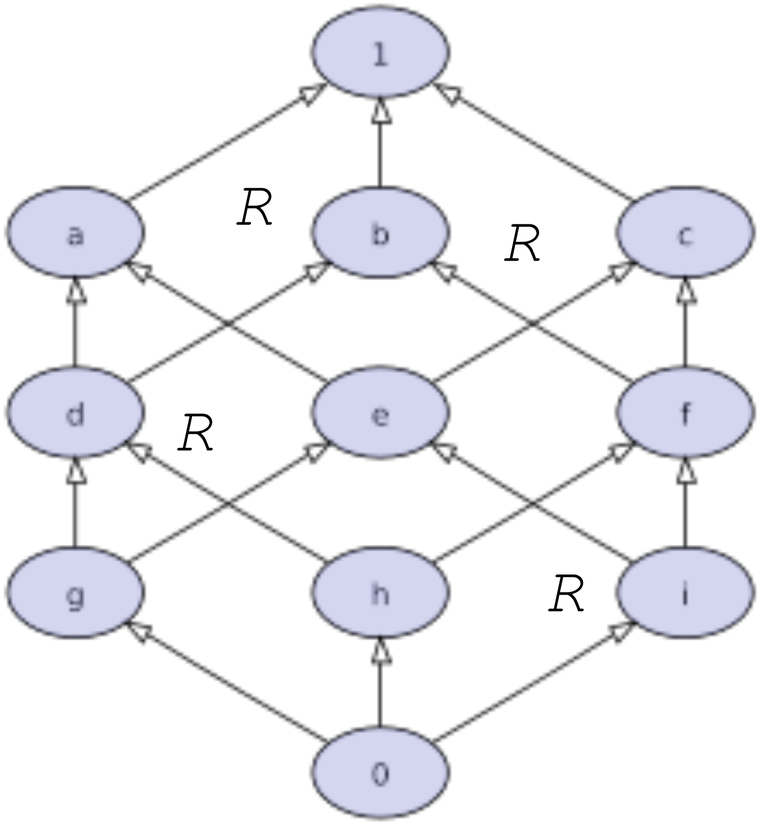
$$b(R) = \left\{ (x, y) \;\middle|\; \begin{array}{l} \forall a, x'.\ x \xrightarrow{a} x' \Rightarrow \exists y'.\ y \xrightarrow{a} y' \wedge (x', y') \in R \\ \forall a, y'.\ y \xrightarrow{a} y' \Rightarrow \exists x'.\ x \xrightarrow{a} x' \wedge (x', y') \in R \end{array} \right\}$$

(b is monotone in the lattice of relations on p, q, and R)



Then `gfp b` is bisimilarity,
which we will take to be observational equivalence.

Recall: Lattices of Relations



Lattice theory is sufficient for defining the appropriate `gfp`:

`A -> Prop` is a complete lattice, and so is `A -> B -> Prop`, etc.

If `b` is `(A -> B -> Prop) -> (A -> B -> Prop)`,

then `gfp b` is `A -> B -> Prop`, and now we have

a relation to work with!

Our study: lattice of relations on states:

```
state -> state -> Prop
```

The game: we want to find the greatest fix point of b

1. As a fix point, it unfolds infinitely:

```
gfp b = b (gfp b) = b (b (gfp b)) = …
```

The game: we want to find the greatest fix point of b

2. As the greatest fix point, it is inhabited for infinite judgements,

   while the least fix point is not.

```
Variant b (R : stream -> stream -> Prop) : stream -> stream -> Prop :=
| bisim_tau s1 s2 : R s1 s2 -> b R (τ s1) (τ s2)
| bisim_beta n s1 s2 : R s1 s2 -> b R ((β n) s1) ((β n) s2).

Definition bisimilar := gfp b.
Definition empty_relation := lfp b.
```

The game: we want to find the greatest fix point of b

Analogy:

`gfp` of the stream functor is infinite `stream`,

while its `lfp` is empty because there is no base case.

```
Variant stream_functor (stream : Type) : Type :=
| τ (s : stream )
| β (n : nat) (s : stream ).

Definition stream := gfp stream_functor.
Definition empty := lfp stream_functor.
```

`gfp` for types is just Rocq's `CoInductive`!

A Note on Notation

`gfp` := greatest fix point := ν (nu)

I will sometimes omit the `b` from `gfp b` when it is clear from context

and say simply “`gfp`” or “the `gfp`”

Finding the `gfp`

There are various ways to construct `gfp b` from `b`.

Perhaps the most famous is a corollary of **Knaster-Tarski**:

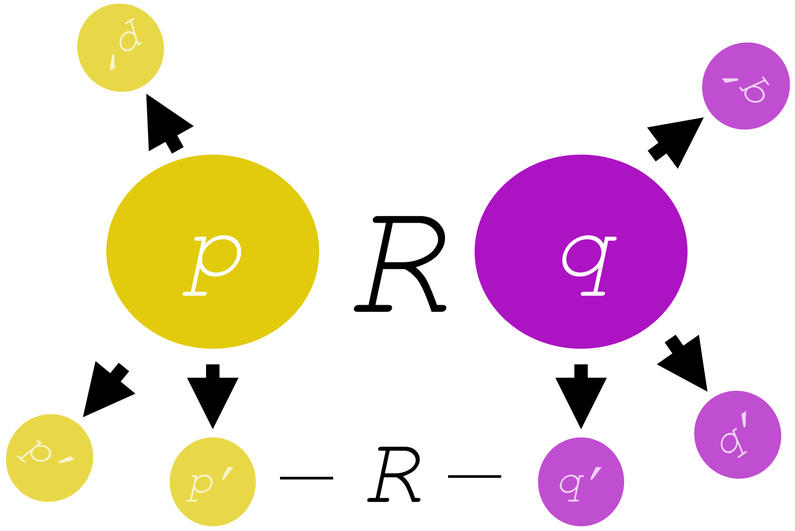
The lattice of fixpoints of `b` is complete, and its top is `gfp b`.

Then the proof method to show membership in `gfp b` is to show membership of anything under it.

Knaster Tarski: Game is to find R that satisfies this diagram.

Or this definition: `R <= bR` (`R` is a post-fix point of `b`).

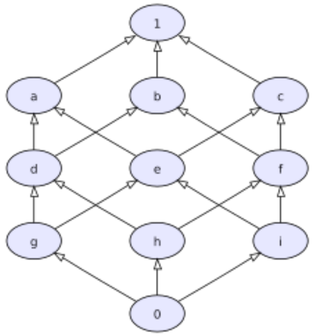
Then you win.



Knaster-Tarski

To show `p, q` are observationally equivalent (bisimiliar), we just need to show they are below *any* (post)fix point of `b`, a.k.a., they are in a *bisimulation.*

gfp b



This works because `gfp b` is the *largest* post-fix point of `b`.

To show `x` is below `gfp b`, just show `x` is below some `R` below the `gfp`.

$$p, q \in gfp\ b \leftrightarrow \exists\ R.\ p, q \in R\ /\backslash\ R\ bisim_b$$

The proof principle:

$$\frac{R \subseteq b(R) \qquad (x,y) \in R}{(x,y) \in \nu b}$$

“Knaster-Tarski Coinduction”

The Principle of Coinduction

$$\frac{x \leqslant y \leqslant by}{x \leqslant \nu b}$$

(we often write it this way)

But… why care about this when we have `cofix` in Rocq?

Problems with Rocq's Native Coinduction

1. Non-compositional
2. Easy to make mistakes with `cofix`
3. Guess the bisimulation candidate *up front*
4. Not amenable to automation

Another big Problem:

1. Non-compositional
2. Easy to make mistakes with `cofix`
3. Guess the bisimulation candidate *up front*
4. Not amenable to automation
5. No support for **up-to** techniques

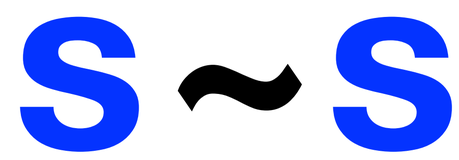
Coinduction Up-to

Motivation: easier proof by coinduction.

`R <= b R` —> `R <= f (b R)` for some `f`

Example: `f x := x \/ id`

This is **“up to reflexivity”**



Could prove manually, but…

Would rather show ~ is reflexive!

Other common up-to functions are symmetry, transitivity, and up-to context.

Coinduction Up-to

Some up-to techniques, like up-to context, are essential for proofs

in sophisticated developments.

```
(* this proof relies on up-to bind of equi, which is nontrivial *)
Lemma Mon_law_2 {A} (m : mon A) : run (Bind m (@Ret A)) == run m.
Proof.
  rewrite run_Bind.
  transitivity (bind (run m) (fun x => now x)).
  - apply bind_cong.
    + reflexivity.
    + intro a. apply run_Ret.
  - apply bind_ret_r.
Qed.
```

```
(1 / 1)
bind (run m) (fun x : A => run (Ret x)) ==
bind (run m) (fun x : A => now x)
```

```
(* this proof relies on up-to bind of equi, which is nontrivial *)
Lemma Mon_law_2 {A} (m : mon A) : run (Bind m (@Ret A)) == run m.
Proof.
  rewrite run_Bind.
  transitivity (bind (run m) (fun x => now x)).
  - apply bind_cong.
    + reflexivity.
    + intro a. apply run_Ret.
  - apply bind_ret_r.
Qed.
```

```
(1 / 2)
run m == run m

Goal 2
(2 / 2)
forall a : A, run (Ret a) == now a
```

Coinduction Wishlist:

1. Compositional
2. Non-syntactic guardedness
3. Nice-to-have: incremental bisimulation
4. Amenable to automation
5. Smooth way to prove soundness of **up-to techniques**

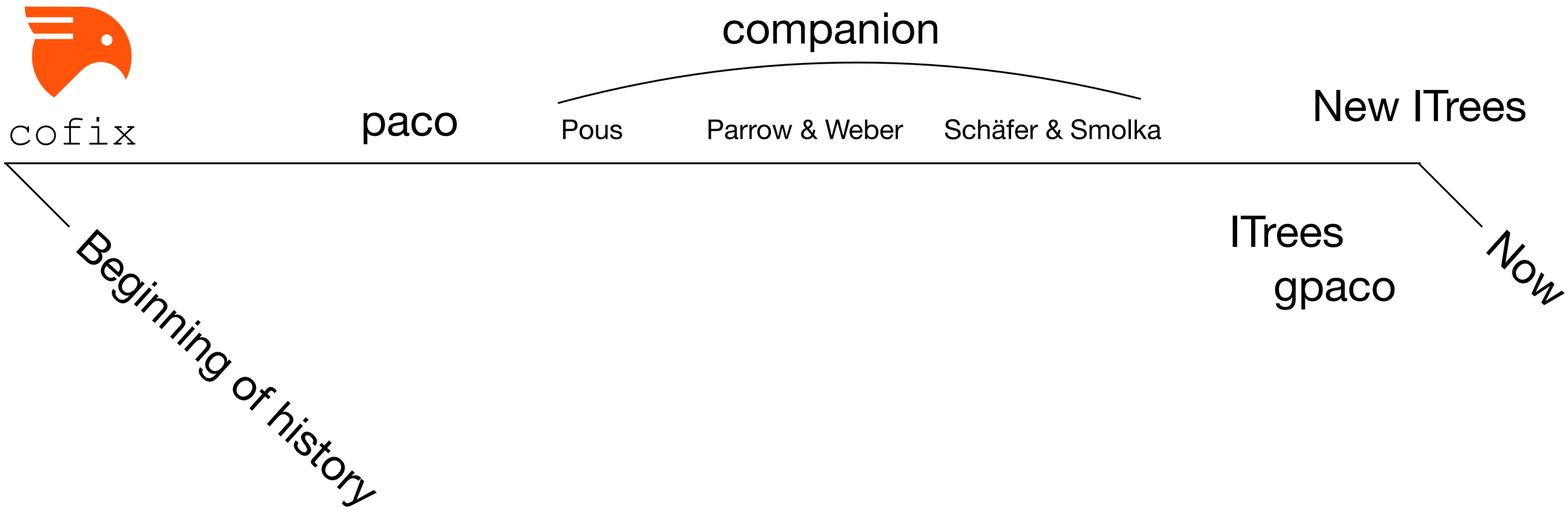
How about just formalizing Knaster-Tarski?

1. Compositional ✅
2. Non-syntactic guardedness ✅
3. Nice-to-have: incremental bisimulation ❌
4. Amenable to automation ✅
5. Smooth way to prove soundness of **up-to techniques** ❌

“Let’s try to do better…”

…

The Pursuit of Happiness in Coinduction



Improvement 1: paco/gpaco

Hur et al. 2013, Zakowski et al. 2020

`paco` (**pa**rametrized **co**induction) formalizes Knaster-Tarski…

$$\mathrm{G}_f(x) \stackrel{\text{def}}{=} \nu y.\ f(x \sqcup y)$$

`G` : the “parameterized” `gfp`

$$\nu f \equiv \mathrm{G}_f(\bot).$$

The Power of Parameterization in Coinductive Proof

Chung-Kil Hur, Microsoft Research, gil@microsoft.com
Georg Neis, MPI-SWS & Saarland University, neis@mpi-sws.org
Derek Dreyer, MPI-SWS, dreyer@mpi-sws.org
Viktor Vafeiadis, MPI-SWS, viktor@mpi-sws.org

…allowing us to find the `gfp` in a more permissive way…

Improvement 1: paco/gpaco

$$\mathrm{G}_f(x) \stackrel{\text{def}}{=} \nu y.\ f(x \sqcup y)$$

$$\nu f \equiv \mathrm{G}_f(\bot).$$

Allows us to find the `gfp` in a more permissive way…

By giving us rules for accumulation and composition.

$$\frac{g_1 \sqsubseteq \mathrm{G}_f(r_1) \qquad r_1 \sqsubseteq r \sqcup g_2 \\ g_2 \sqsubseteq \mathrm{G}_f(r_2) \qquad r_2 \sqsubseteq r \sqcup g_1}{g_1 \sqcup g_2 \sqsubseteq \mathrm{G}_f(r)}$$

COMPOSE, for combining theorems

$$y \sqsubseteq \mathrm{G}_f(x) \iff y \sqsubseteq \mathrm{G}_f(x \sqcup y).$$

ACCUMULATE, for adding knowledge during a proof

$$\mathrm{G}_f(x) \equiv f(x \sqcup \mathrm{G}_f(x)).$$

UNFOLD, for exposing the monotone function `f`

Improvement 1: paco/gpaco

Hur et al. 2013, Zakowski et al. 2020

`gpaco` (**g**eneralized **paco)** exploits *compatible enhancements* to the `gfp`

for **up-to** reasoning.

$$\hat{G}_f^{bclo}\ r\ g \stackrel{\text{def}}{=} bclo^*(r \sqcup G_{f \circ bclo^*}\ (r \sqcup g))$$

An Equational Theory for Weak Bisimulation via Generalized Parameterized Coinduction

Yannick Zakowski
University of Pennsylvania
Philadelphia, PA, USA

Paul He
University of Pennsylvania
Philadelphia, PA, USA

Chung-Kil Hur
Seoul National University
Seoul, Republic of Korea

Steve Zdancewic
University of Pennsylvania
Philadelphia, PA, USA

$$\frac{g_1 \sqsubseteq \mathrm{G}_f(r_1) \qquad r_1 \sqsubseteq r \sqcup g_2 \\ g_2 \sqsubseteq \mathrm{G}_f(r_2) \qquad r_2 \sqsubseteq r \sqcup g_1}{g_1 \sqcup g_2 \sqsubseteq \mathrm{G}_f(r)}$$

$$y \sqsubseteq \mathrm{G}_f(x) \iff y \sqsubseteq \mathrm{G}_f(x \sqcup y).$$

$$\nu f \equiv \mathrm{G}_f(\bot). \qquad \mathrm{G}_f(x) \stackrel{\text{def}}{=} \nu y.\ f(x \sqcup y) \qquad \mathrm{G}_f(x) \equiv f(x \sqcup \mathrm{G}_f(x)).$$

Paco stats:

1. Compositional ✅
2. Non-syntactic guardedness ✅
3. Nice-to-have: incremental bisimulation ✅
4. Amenable to automation ✅
5. Smooth way to prove soundness of **up-to techniques** 😳

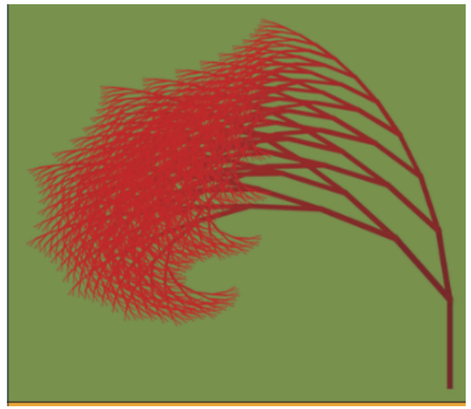
… but <u>much</u> better than cofix!

From paco, Interaction Trees (ITrees)

Interaction Trees

Idea: Abstract away all coinductive proof, and do reasoning modulo an equational theory.

BENJAMIN C. PIERCE, University of Pennsylvania, USA
STEVE ZDANCEWIC, University of Pennsylvania, USA



```
Variant itreeF (itree : Type) :=
| RetF (r : R)
| TauF (t : itree)
| VisF {X : Type} (e : E X) (k : X -> itree)
.
```

`itree` is the `CoInductive itreeF`.

$$\hat{G}_f^{bclo}\ r\ g \stackrel{\text{def}}{=} bclo^*(r \sqcup G_{f \circ bclo^*}\ (r \sqcup g))$$

$$\frac{\begin{array}{ll} g_1 \sqsubseteq \mathrm{G}_f(r_1) & r_1 \sqsubseteq r \sqcup g_2 \\ g_2 \sqsubseteq \mathrm{G}_f(r_2) & r_2 \sqsubseteq r \sqcup g_1 \end{array}}{g_1 \sqcup g_2 \sqsubseteq \mathrm{G}_f(r)}$$

$$y \sqsubseteq \mathrm{G}_f(x) \iff y \sqsubseteq \mathrm{G}_f(x \sqcup y).$$

$$\nu f \equiv \mathrm{G}_f(\bot). \quad \mathrm{G}_f(x) \stackrel{\text{def}}{=} \nu y.\ f(x \sqcup y) \quad \mathrm{G}_f(x) \equiv f(x \sqcup \mathrm{G}_f(x)).$$

ITrees: A Coinductive *Data Structure*

```
(** The type [itree] is defined as the final coalgebra ("greatest
    fixed point") of the functor [itreeF]. *)
Variant itreeF (itree : Type) :=
| RetF (r : R)
| TauF (t : itree)
| VisF {X : Type} (e : E X) (k : X -> itree)
.

(** We define non-recursive types such as [itreeF] using the [Variant]
    command. The main practical difference from [Inductive] is that
    [Variant] does not generate any induction schemes (which are
    unnecessary). *)

CoInductive itree : Type := go
{ _observe : itreeF itree }.
```

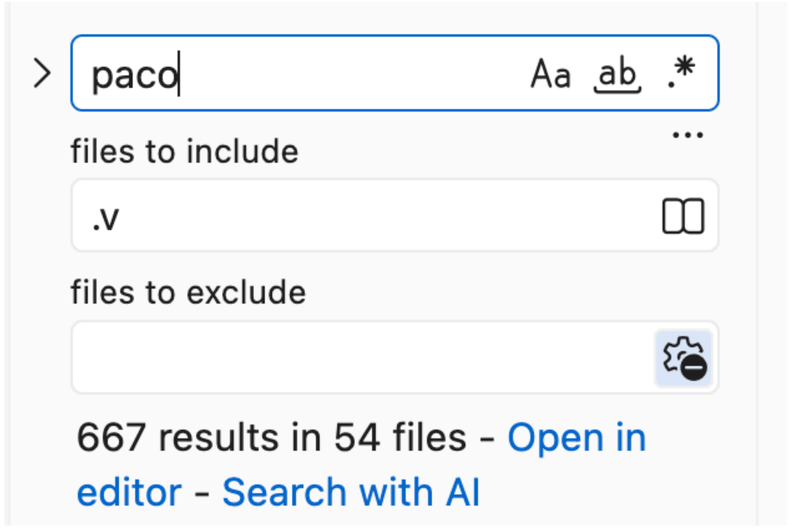
ITree: A data structure for:

1. Modeling infinite and impure computation

2. Hiding the low-level coinductive reasoning one would do with paco.

3. Extractable execution for verified software

Paco is Pervasive in ITrees, and is used to define bisimilarity




```
Inductive eqitF (b1 b2: bool) vclo (sim : itree E R1 -> itree E R2 -> Prop) :
  itree' E R1 -> itree' E R2 -> Prop :=
| EqRet r1 r2
    (REL: RR r1 r2):
   eqitF b1 b2 vclo sim (RetF r1) (RetF r2)
| EqTau m1 m2
    (REL: sim m1 m2):
   eqitF b1 b2 vclo sim (TauF m1) (TauF m2)
| EqVis {u} (e : E u) k1 k2
    (REL: forall v, vclo sim (k1 v) (k2 v) : Prop):
   eqitF b1 b2 vclo sim (VisF e k1) (VisF e k2)
| EqTauL t1 ot2
    (CHECK: b1)
    (REL: eqitF b1 b2 vclo sim (observe t1) ot2):
   eqitF b1 b2 vclo sim (TauF t1) ot2
| EqTauR ot1 t2
    (CHECK: b2)
    (REL: eqitF b1 b2 vclo sim ot1 (observe t2)):
   eqitF b1 b2 vclo sim ot1 (TauF t2)
.
Hint Constructors eqitF : itree.
```

b

```
Definition eqit b1 b2 : itree E R1 -> itree E R2 -> Prop :=
  paco2 (eqit_ b1 b2 id) bot2.
```

gfp b

… but paco has some weaknesses

Weaknesses of Paco

```
Lemma eqitC_wcompat b1 b2 vclo      Gil Hur, 7 years ago via  PR #128 • WIP
      (MON: monotone2 vclo)
      (CMP: compose (eqitC b1 b2) vclo <3= compose vclo (eqitC b1 b2)):
  wcompatible2 (@eqit_ E R1 R2 RR b1 b2 vclo) (eqitC b1 b2).
Proof with eauto with paco itree.
  econstructor; [ eauto with paco itree | ].
  intros. destruct PR.
  punfold EQVl. punfold EQVr. unfold_eqit.
  hinduction REL before r; intros; clear t1' t2'.
  - remember (RetF r1) as x.
    hinduction EQVl before r; intros; subst; try inv Heqx; [ | eauto with itree ].
    remember (RetF r3) as y.
    hinduction EQVr before r; intros; subst; try inv Heqy...
  - remember (TauF m1) as x.
    hinduction EQVl before r; intros; subst; try inv Heqx; try inv CHECK; [ | eauto with itree ].
    remember (TauF m3) as y.
    hinduction EQVr before r; intros; subst; try inv Heqy; try inv CHECK; [ | eauto with itree ].
    pclearbot. econstructor. gclo.
    econstructor; eauto with paco.
  - remember (VisF e k1) as x.
    hinduction EQVl before r; intros; try discriminate Heqx; [ inv_Vis | eauto with itree ].
    remember (VisF e k3) as y.
    hinduction EQVr before r; intros; try discriminate Heqy; [ inv_Vis | eauto with itree ].
    econstructor. intros. pclearbot.
    eapply MON.
    + apply CMP. econstructor...
    + intros. apply gpaco2_clo, PR.
  - remember (TauF t1) as x.
    hinduction EQVl before r; intros; subst; try inv Heqx; try inv CHECK; [ | eauto with itree ].
    pclearbot. punfold REL...
  - remember (TauF t2) as y.
    hinduction EQVr before r; intros; subst; try inv Heqy; try inv CHECK; [ | eauto with itree ].
    pclearbot. punfold REL...
Qed.
```

Long compatibility proofs

Weaknesses of Paco

```
Lemma eqit_clo_trans b1 b2 vclo
      (MON: monotone2 vclo)
      (CMP: compose (eqitC b1 b2) vclo <3= compose vclo (eqitC b1 b2)):
  eqit_trans_clo b1 b2 false false <3= gupaco2 (eqit_ RR b1 b2 vclo) (eqitC b1 b2).
Proof.
  intros. destruct PR. gclo. econstructor; eauto with paco.
Qed.
```

Lemmas reasoning about compatibility are hard to read and maintain

Weaknesses of Paco

```
Lemma state_independent : forall {S R} (t:itree (stateE S) R)
                           (H: NoGets t),
    forall s s', ('(s,x) <- interpret_state t s ;; ret x) ≅ ('(s,x) <- interpret_state t s' ;; ret x).
Proof.
  intros S R.
  ginit. gcofix CIH.
  intros t H0 s s'.
  rewrite (itree_eta (interpret_state t s)).
  rewrite (itree_eta (interpret_state t s')).
  rewrite !unfold_interpret_state.
  unfold interpret_stateF.
  punfold H0. repeat red in H0.
  destruct (observe t); cbn.
  - rewrite !bind_ret_l. gstep. econstructor. eauto.
  - rewrite !bind_tau. gstep. econstructor.
    gbase. eapply CIH.
    inversion H0. subst. pclearbot. assumption.
  - destruct e; cbn.
    + (* e is Get, which is ruled out by the NoGets predicate *) inversion H0.
    + rewrite !bind_tau.
      gstep. econstructor. gbase. eapply CIH.
      inversion H0. ddestruction. pclearbot. assumption.
Qed.
```

Lots of individual tactics make it a bit tricky to use and automate

(6 paco-specific tactics in this proof)

Can we do better?

Paco is a complete theory of up-to coinduction: Problem is only in usability.

Main problems:

1. Compatibility proofs can get long,
2. Proofs with gpaco closures can get confusing and use different techniques,
3. Lots of tactics to remember (4-8+ unique paco tactics per proof)
4. Minor wart: “arity hell:” each paco operator is re-defined for different arities,

   e.g., `paco1`, `paco2`, up to to `paco16`…

   which lags my autocomplete server for 8 seconds per keystroke :(

(and makes automation harder to write)

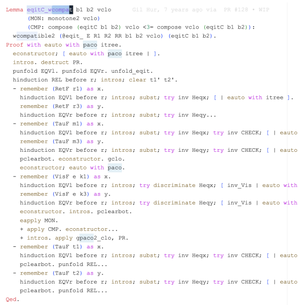
So, our goal is to find a theory that gives “nicer” proofs, is easier to maintain and teach, etc.

Can we do better?

One other theoretical wart:

To do coinduction up-to a function `f`, we must fix `f` and prove that it is compatible

(these are the proofs that can be very long and tedious).



```
Lemma eqitC_wcompatible2 b1 b2 vclo
      (MON: monotone2 vclo)
      (CMP: compose (eqitC b1 b2) vclo <3= compose vclo (eqitC b1 b2)):
  wcompatible2 (@eqit_ E R1 R2 RR b1 b2 vclo) (eqitC b1 b2).
Proof with eauto with paco itree.
  econstructor; [ eauto with paco itree | ].
  intros. destruct PR.
  punfold EQV1. punfold EQVr. unfold_eqit.
  hinduction REL before r; intros; clear t1' t2'.
  - remember (RetF r1) as x.
    hinduction EQV1 before r; intros; subst; try inv Heqx; [ | eauto with itree ].
    remember (RetF r3) as y.
    hinduction EQVr before r; intros; subst; try inv Heqy...
  - remember (TauF m1) as x.
    hinduction EQV1 before r; intros; subst; try inv Heqx; try inv CHECK; [ | eauto with itree ].
    remember (TauF m3) as y.
    hinduction EQVr before r; intros; subst; try inv Heqy; try inv CHECK; [ | eauto with itree ].
    pclearbot. econstructor. gclo.
    econstructor; eauto with paco.
  - remember (VisF e k1) as x.
    hinduction EQV1 before r; intros; try discriminate Heqx; [ inv_Vis | eauto with itree ].
    remember (VisF e k3) as y.
    hinduction EQVr before r; intros; try discriminate Heqy; [ inv_Vis | eauto with itree ].
    econstructor. intros. pclearbot.
    eapply MON.
    + apply CMP. econstructor...
    + intros. apply upaco2_clo. PR.
  - remember (TauF t1) as x.
    hinduction EQV1 before r; intros; subst; try inv Heqx; try inv CHECK; [ | eauto with itree ].
    pclearbot. punfold REL...
  - remember (TauF t2) as y.
    hinduction EQVr before r; intros; subst; try inv Heqy; try inv CHECK; [ | eauto with itree ].
    pclearbot. punfold REL...
Qed.
```

… several such proofs in the itrees development along variations of bisimilarity

It would be great if we didn’t have to think so much about which enhancements to pick…

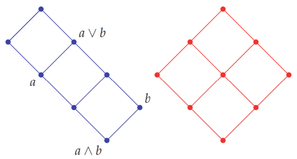
Attempt 1: Enhanced Coinduction

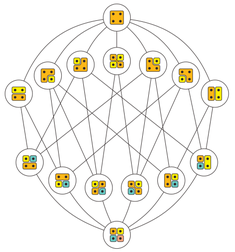
Coinduction All the Way Up

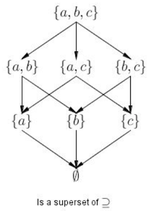
Damien Pous *

Université de Lyon, CNRS, ENS de Lyon, UCB Lyon 1, LIP

Damien.Pous@ens-lyon.fr







Enhanced coinduction, up to the companion

The companion of `b` is `b`'s greatest compatible function"

No more need to pick an `f`: just pick the companion every time!

- The companion provides a proof system for up-to coinduction

$$\frac{y \leqslant t\bot}{y \leqslant \nu b}\ \textsc{Init} \qquad \frac{y \leqslant x}{y \leqslant tx}\ \textsc{Done}$$

$$\frac{y \leqslant ftx \qquad f \leqslant t}{y \leqslant tx}\ \textsc{Up to } f \qquad \frac{y \leqslant bt(y \vee x)}{y \leqslant tx}\ \textsc{CoInd}$$

- It is a bit friendlier to use than paco, but still has some warts.

- In particular, proving up-to techniques valid sometimes takes a circuitous route through a "second order companion," which can be confusing and tedious.

:P

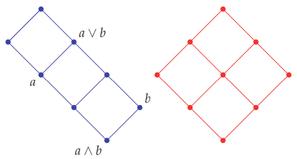
Attempt 2: Wait until Enhanced Coinduction got better

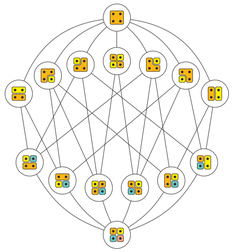
Coinduction All the Way Up

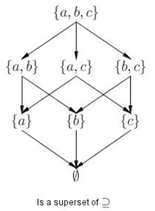
Damien Pous *

Université de Lyon, CNRS, ENS de Lyon, UCB Lyon 1, LIP

Damien.Pous@ens-lyon.fr







Speeding ahead

Parrow–Weber: The `gfp` as the end of a classical transfinite chain

THE LARGEST RESPECTFUL FUNCTION

JOACHIM PARROW, TJARK WEBER

Uppsala University, Sweden

ABSTRACT. Respectful functions were introduced by Sangiorgi as a compositional tool to formulate short and clear bisimulation proofs. Usually, the larger the respectful function, the easier the bisimulation proof. In particular the largest respectful function, defined as the pointwise union of all respectful functions, has been shown to be very useful. We here provide an explicit and constructive characterization of it.

A new way to find the `gfp`: through transfinite iteration

Interesting, but classical, so difficult to write constructive proofs

$$gfp\ b = b^{\lambda}(\top)$$

Schäfer–Smolka: The same, but constructively

Tower Induction and Up-To Techniques for CCS with Fixed Points

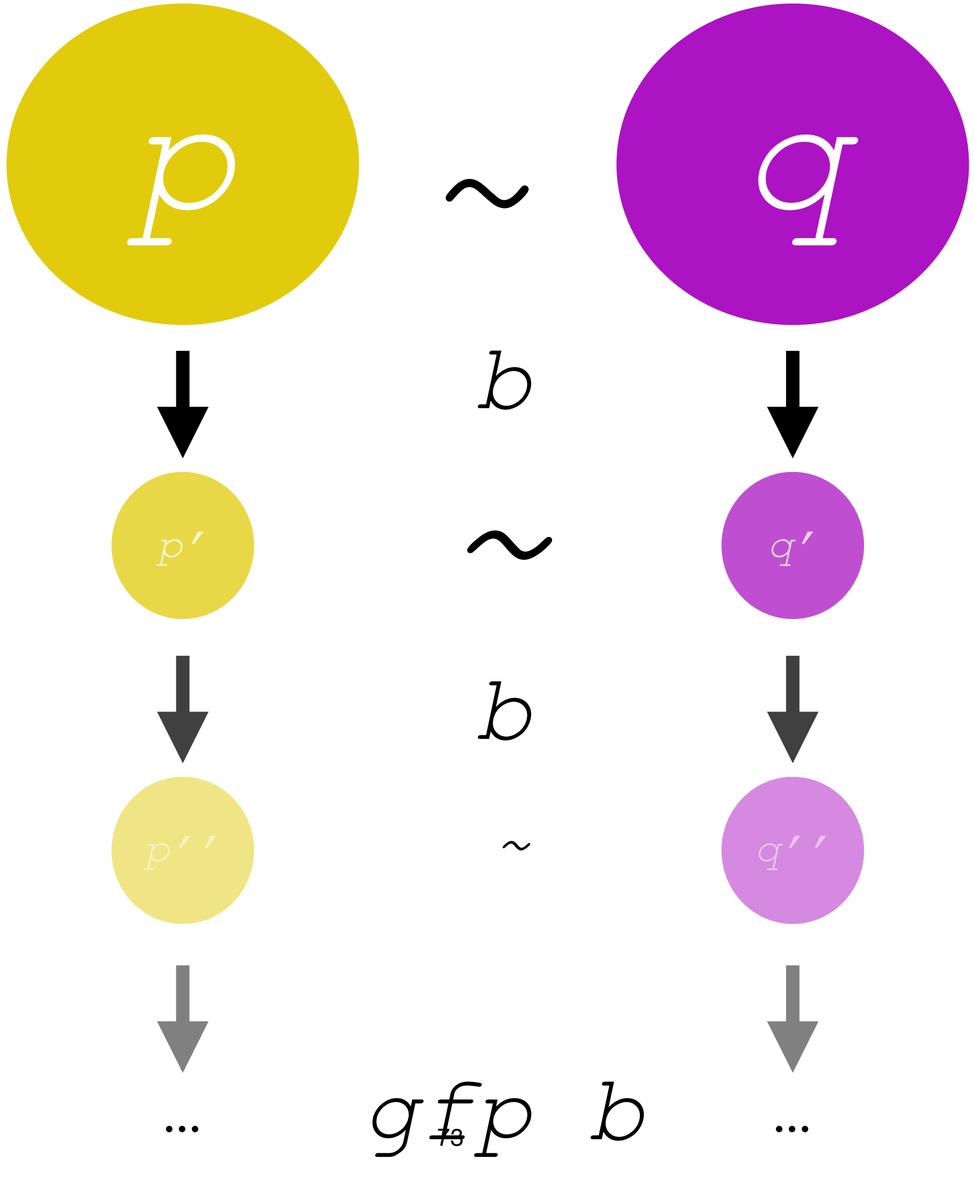
Steven Schäfer and Gert Smolka

Saarland University,
Saarbrücken, Germany
{schaefer, smolka}@ps.uni-saarland.de

Constructively?

The Tower

The Tower



The Tower

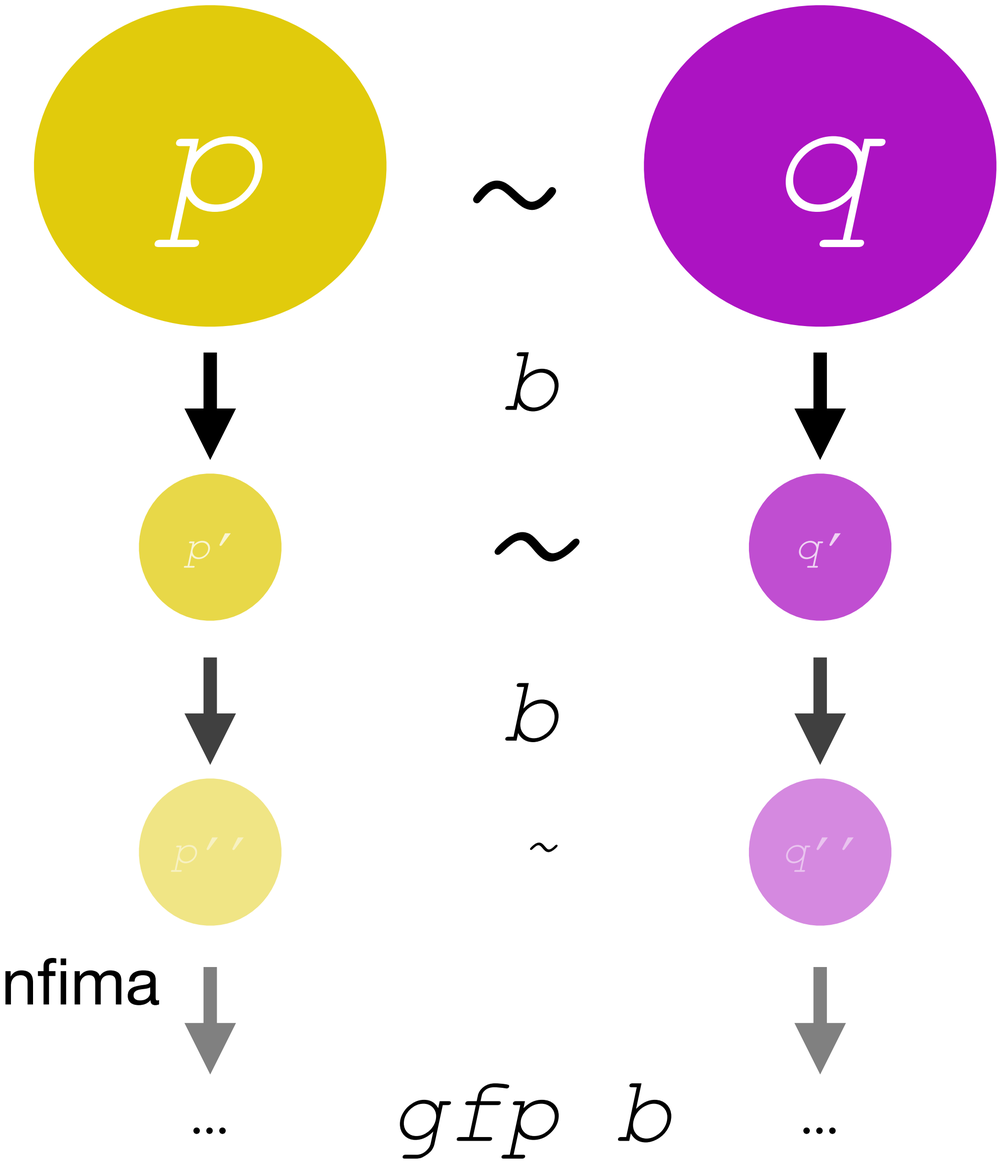
“Start at the `top` and `b` down to the `gfp`”

```
Inductive C: X -> Prop :=
| Cb: forall x, C x -> C (b x)
| Cinf: forall T, T <= C -> C (inf T).
```

The tower is closed under `b`

The tower is closed under infima

(greatest lower bounds)

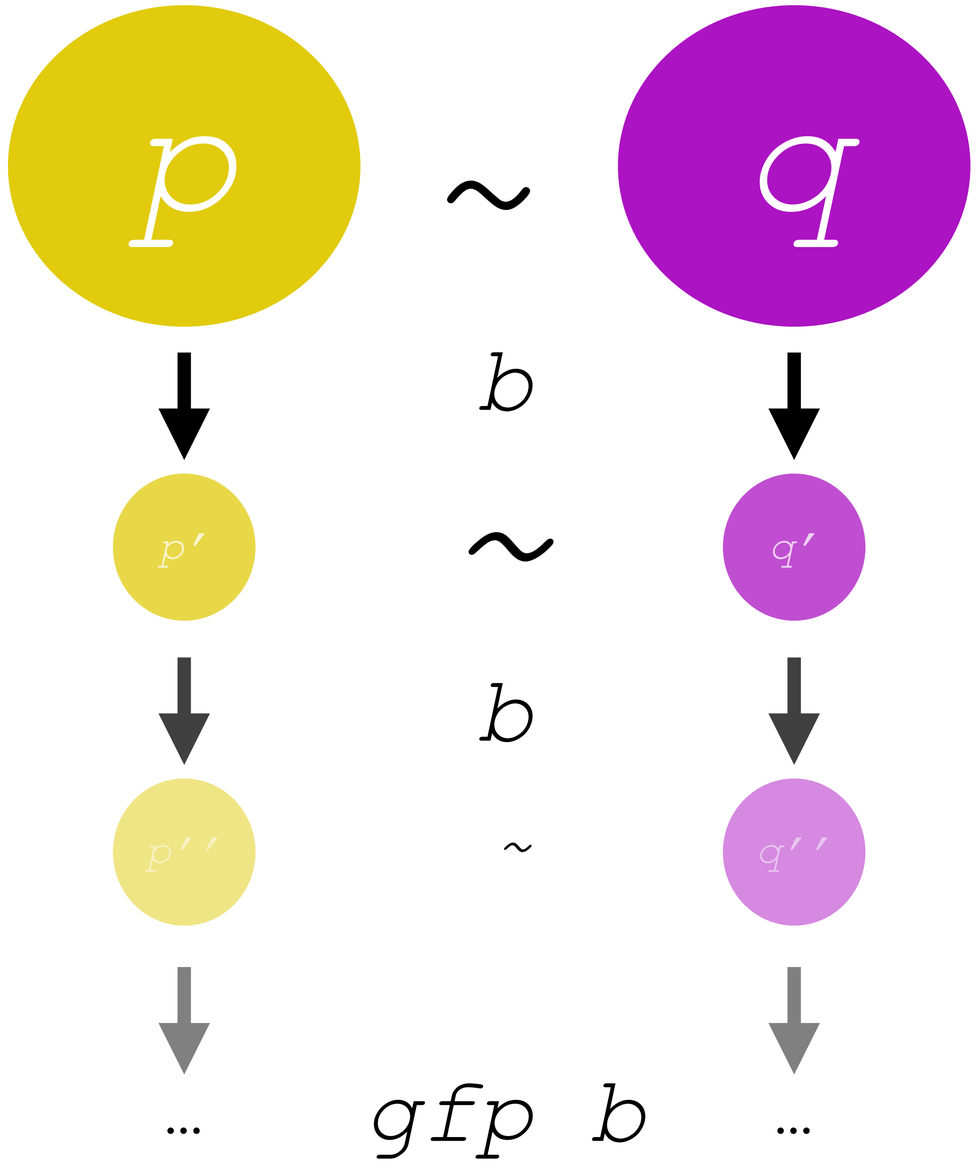


The Tower

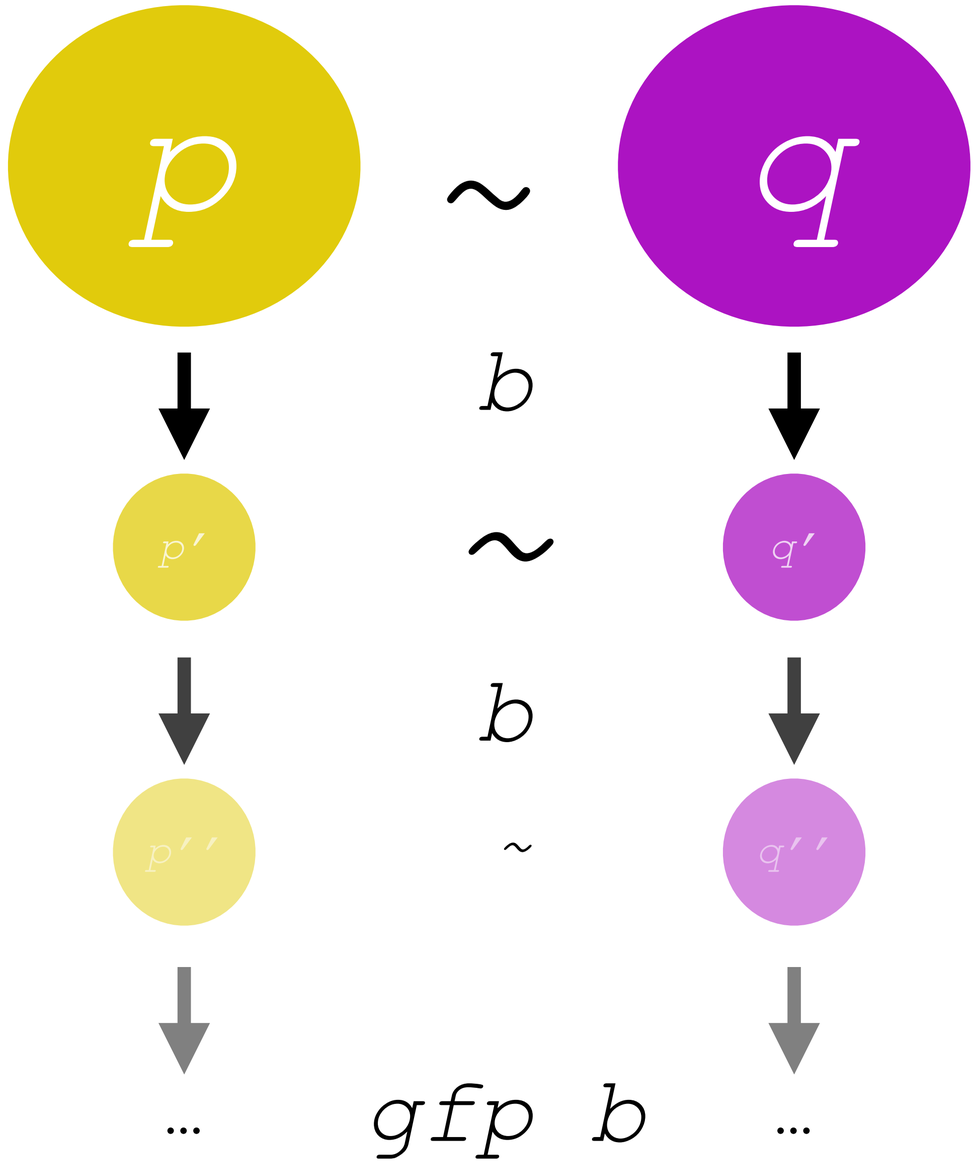
Transfiniteness recovered!

```
Inductive C: X -> Prop :=
| Cb: forall x, C x -> C (b x)
| Cinf: forall T, T <= C -> C (inf T).
```

$$gfp\ b = inf\ C = b^{\lambda}(\top)$$



How the tower works:



“Start at the `top` and `b` down to the `gfp`”

```
inf {} = T
```

b T

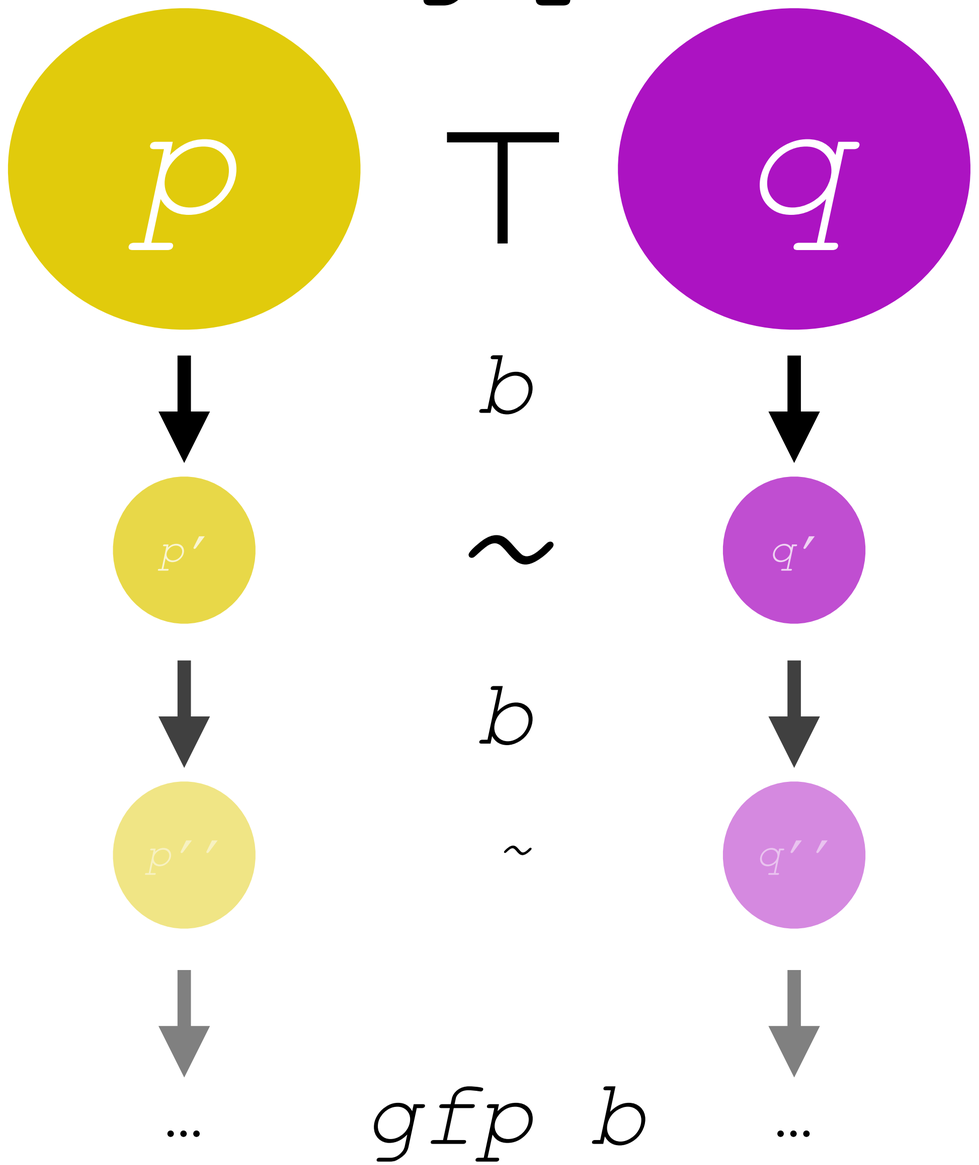
*b*² T

*b*³ T

…

inf (*b*ⁿ T)

```
Inductive C: X -> Prop :=
| Cb: forall x, C x -> C (b x)
| Cinf: forall T, T <= C -> C (inf T).
```



“Start at the `top` and `b` down to the `gfp`”

```
inf {} = T
```

$b\ \top$

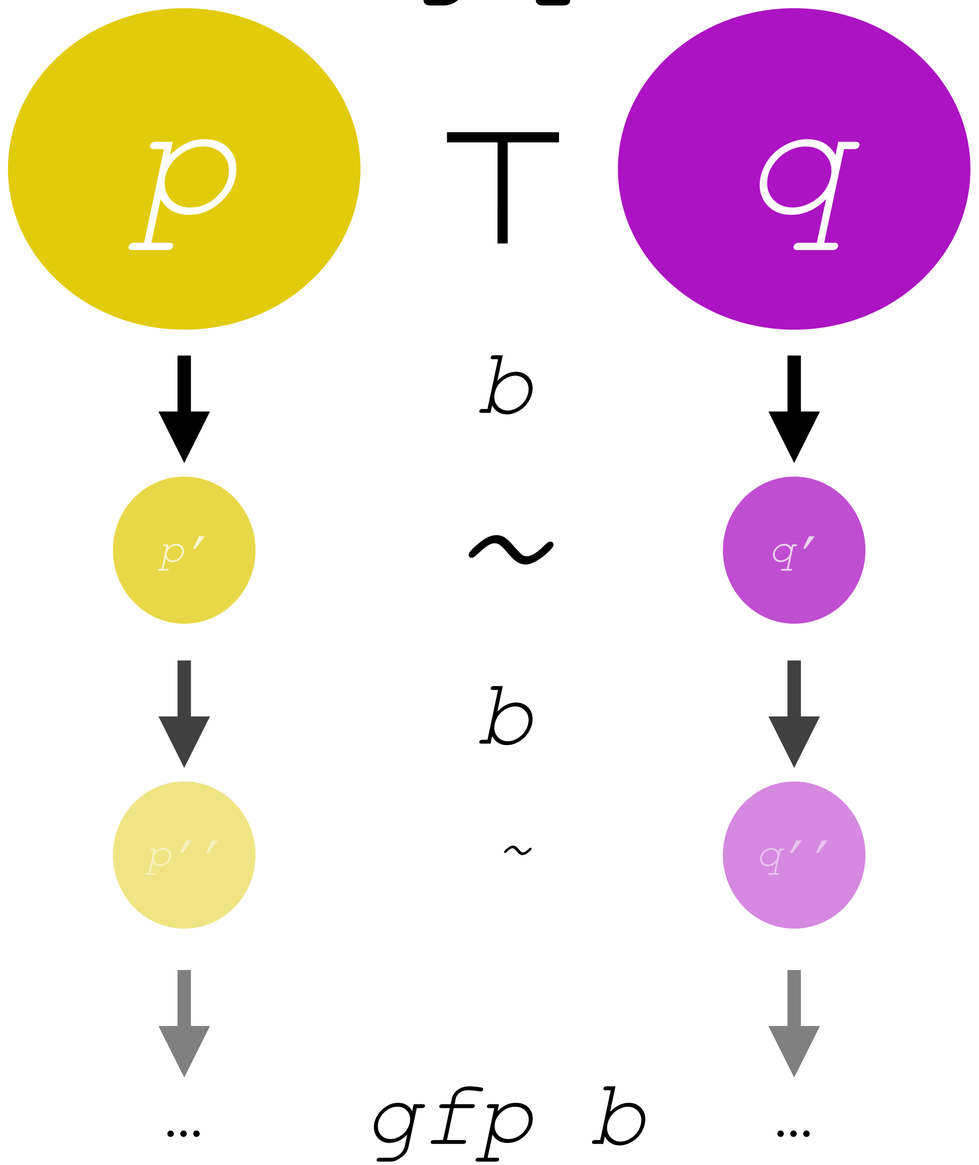
$b^2\ \top$

$b^3\ \top$

…

$inf\ (b^n\ \top)$

```
Inductive C: X -> Prop :=
| Cb: forall x, C x -> C (b x)
| Cinf: forall T, T <= C -> C (inf T).
```

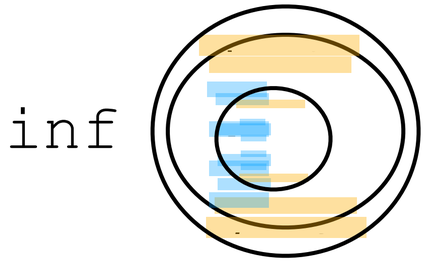


“Start at the `top` and `b` down to the `gfp`”

b, inf …

b, inf …

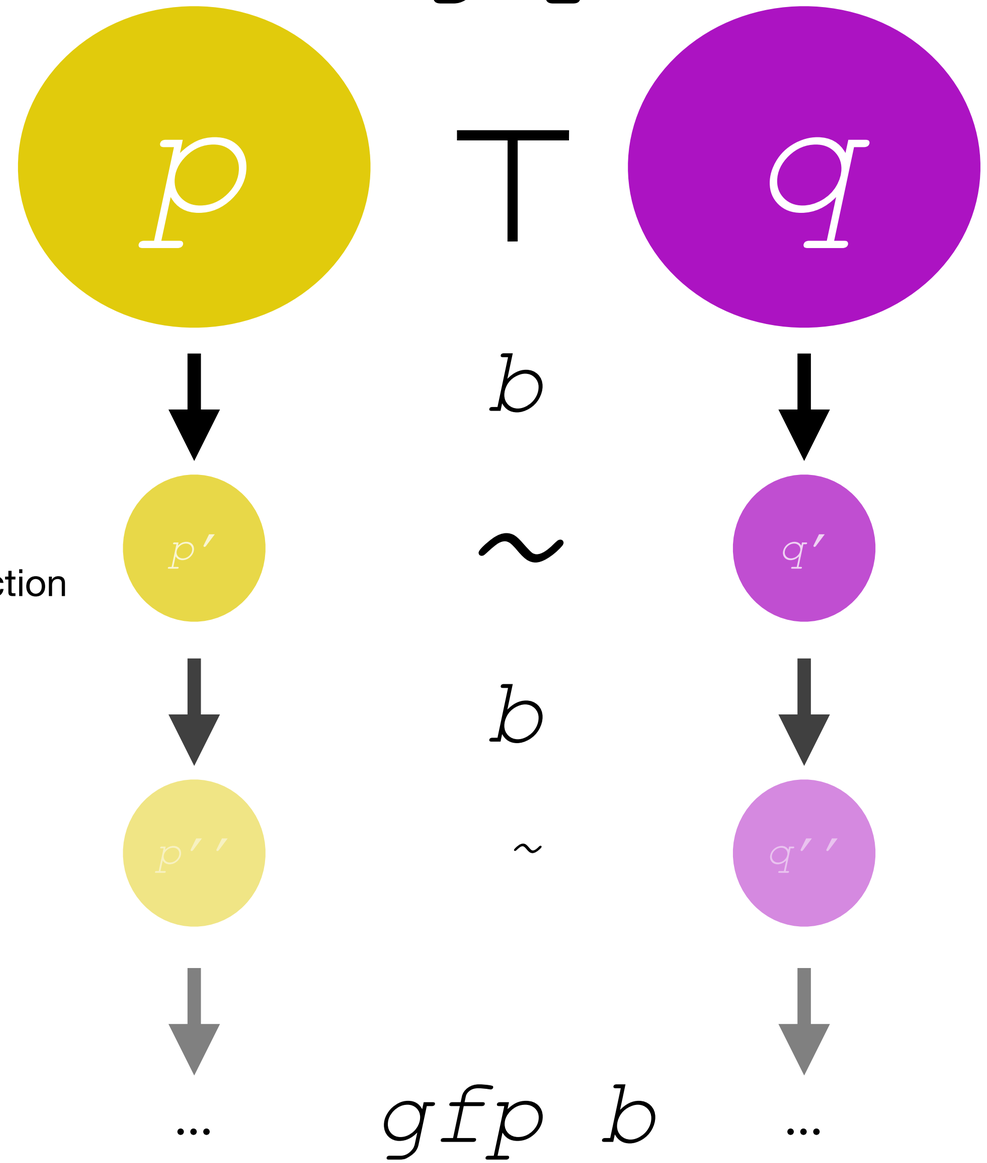
b, inf …



Open: prove this is the same

as Parrow & Weber’s ordinal construction

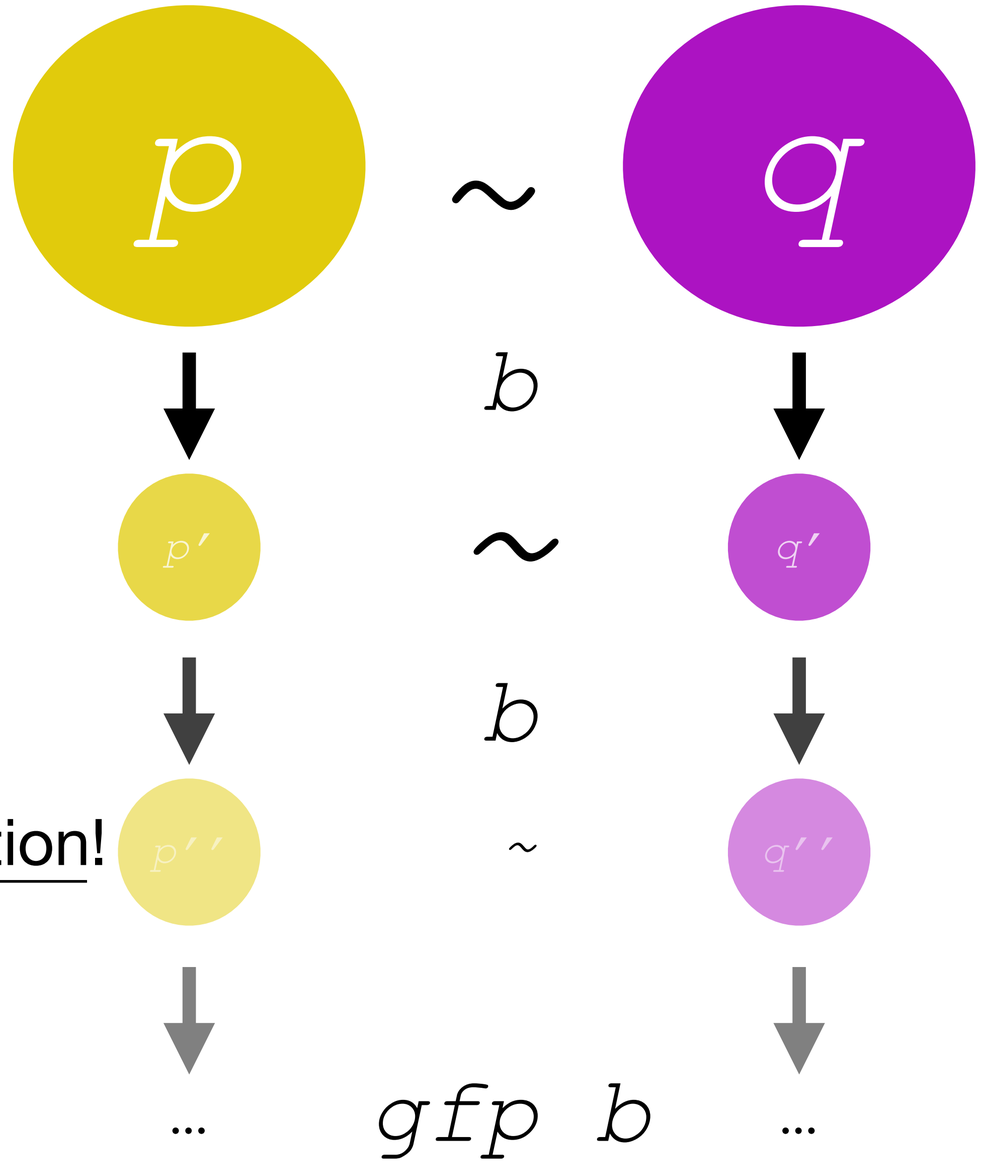
= inf C = gfp b




```
Inductive C: X -> Prop :=
| Cb: forall x, C x -> C (b x)
| Cinf: forall T, T <= C -> C (inf T).
```

The Tower: The best part

```
Inductive C: X -> Prop :=
| Cb: forall x, C x -> C (b x)
| Cinf: forall T, T <= C -> C (inf T).
```



We get an inductive proof principle to do coinduction!

And indeed to do more…

Up-to and coinductive proofs with the tower

To prove `f` is a sound up-to of the tower,

prove `f` holds of every element of `C` by induction on `C`.

```
Inductive C: X -> Prop :=
| Cb: forall x, C x -> C (b x)
| Cinf: forall T, T <= C -> C (inf T).
```

This *tower induction* technique is the single proof method for both up-to proofs and coinduction.

```
H :
  forall (t1 : itree E U1) (t2 : itree E U2)
    (k1 : U1 -> itree E R1) (k2 : U2 -> itree E R2)
    (UU : U1 -> U2 -> Prop),
  elem c U1 U2 UU t1 t2 ->
  (forall (u1 : U1) (u2 : U2),
   UU u1 u2 -> elem c R1 R2 RR (k1 u1) (k2 u2)) ->
  elem c R1 R2 RR (ITree.bind t1 k1) (ITree.bind t2 k2)
(1 / 1) ──────────────────────────────────────────────
eqitF RR b1 b2 (elem c R1 R2 RR) (⊙ (ITree.bind t1 k1))
  (⊙ (ITree.bind t2 k2))
```

Up-to and coinductive proofs with the tower

This *tower induction* technique is the single proof method for both up-to proofs and coinduction.

```
Inductive C: X -> Prop :=
| Cb: forall x, C x -> C (b x)
| Cinf: forall T, T <= C -> C (inf T).
```

Indeed, to prove two elements are related by the gfp, you can use tower induction:

1. Turn the relation into a predicate `P`
2. Prove `P` holds for all elements of the tower by tower induction
3. The `gfp` is in the tower, so `P` holds for the `gfp`.

Up-to and coinductive proofs with the tower

```
Inductive C: X -> Prop :=
| Cb: forall x, C x -> C (b x)
| Cinf: forall T, T <= C -> C (inf T).
```

Indeed, to prove two elements are related by the gfp, you can use tower induction:

1. Turn the relation into a predicate `P`
2. Prove `P` holds for all elements of the tower by tower induction
3. The `gfp` is in the tower, so `P` holds for the `gfp`.

This can be done automatically

The base case is almost always trivial

“I can’t believe it’s not coinduction!”

```
Inductive C: X -> Prop :=
| Cb: forall x, C x -> C (b x)
| Cinf: forall T, T <= C -> C (inf T).
```

If we have no base case, we just have to prove stability under `b`…

This looks just like coinduction!

```
c : Chain (eqit_mon b1 b2)
CIH : forall x : itree E R, elem c R R RR x x

(1/1) ──────────────────────────────────────────────────────────────
forall x : itree E R, eqitF RR b1 b2 (elem c R R RR) (⊙ x) (⊙ x)
```

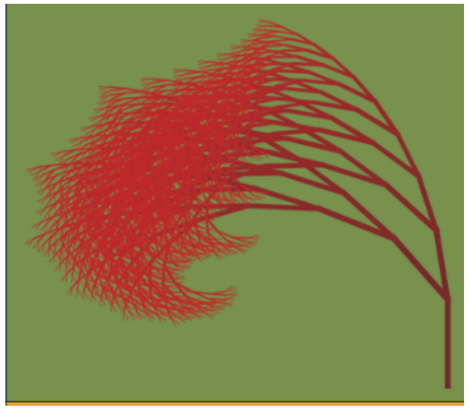
Tower stats (my claim):

1. Compositional ✅
2. Non-syntactic guardedness ✅
3. Nice-to-have: incremental bisimulation ✅
4. Amenable to automation ✅ (very few tactics, all with uniform behavior)
5. Smooth way to prove soundness of **up-to techniques** ✅

(also) by *induction!*

So we have a nice library… now what?

Theory meets Practice*: Roger ports ITrees from paco to the tower



```
Lemma eqitC_wcompat b1 b2 vclo      Gil Hur, 7 years ago via  PR #128 • WIP
      (MON: monotone2 vclo)
      (CMP: compose (eqitC b1 b2) vclo <3= compose vclo (eqitC b1 b2)):
  wcompatible2 (@eqit_ E R1 R2 RR b1 b2 vclo) (eqitC b1 b2).
Proof with eauto with paco itree.
  econstructor; [ eauto with paco itree | ].
  intros. destruct PR.
  punfold EQV1. punfold EQVr. unfold_eqit.
  hinduction REL before r; intros; clear t1' t2'.
  - remember (RetF r1) as x.
    hinduction EQV1 before r; intros; subst; try inv Heqx; [ | eauto with itree ].
    remember (RetF r3) as y.
    hinduction EQVr before r; intros; subst; try inv Heqy...
  - remember (TauF m1) as x.
    hinduction EQV1 before r; intros; subst; try inv Heqx; try inv CHECK; [ | eauto with itree ].
    remember (TauF m3) as y.
    hinduction EQVr before r; intros; subst; try inv Heqy; try inv CHECK; [ | eauto with itree ].
    pclearbot. econstructor. gclo.
    econstructor; eauto with paco.
  - remember (VisF e k1) as x.
    hinduction EQV1 before r; intros; try discriminate Heqx; [ inv_Vis | eauto with itree ].
    remember (VisF e k3) as y.
    hinduction EQVr before r; intros; try discriminate Heqy; [ inv_Vis | eauto with itree ].
    econstructor. intros. pclearbot.
    eapply MON.
    + apply CMP. econstructor...
    + intros. apply gpaco2_clo, PR.
  - remember (TauF t1) as x.
    hinduction EQV1 before r; intros; subst; try inv Heqx; try inv CHECK; [ | eauto with itree ].
    pclearbot. punfold REL...
  - remember (TauF t2) as y.
    hinduction EQVr before r; intros; subst; try inv Heqy; try inv CHECK; [ | eauto with itree ].
    pclearbot. punfold REL...
Qed.
```

* of other theory

Improvements Anticipated…

Improvements Found?

Was this experiment a success?
What did it tell us?

Some observations

Paco

```
Inductive eqit_Leaf_bind_clo b1 b2 (r : itree E R -> itree E S -> Prop) :
  itree E R -> itree E S -> Prop :=
| pbc_intro_h U1 U2 (RU : U1 -> U2 -> Prop)
              (t1 : itree E U1) (t2 : itree E U2)
              (k1 : U1 -> itree E R) (k2 : U2 -> itree E S)
              (EQV: eqit RU b1 b2 t1 t2)
              (REL: forall u1 u2,
                  u1 ∈ t1 -> u2 ∈ t2 -> RU u1 u2 ->
                  r (k1 u1) (k2 u2))
    : eqit_Leaf_bind_clo b1 b2 r
        (ITree.bind t1 k1) (ITree.bind t2 k2)
.
Hint Constructors eqit_Leaf_bind_clo : itree.

Lemma eqit_Leaf_clo_bind  (RS : R -> S -> Prop) b1 b2 vclo
      (MON: monotone2 vclo)
      (CMP: compose (eqitC RS b1 b2) vclo <3= compose vclo (eqitC RS b1 b2))
      (ID: id <3= vclo):
  eqit_Leaf_bind_clo b1 b2 <3= gupaco2 (eqit_ RS b1 b2 vclo) (eqitC RS b1 b2).
Proof.
  gcofix CIH. intros. destruct PR.
  guclo eqit_clo_trans.
  econstructor; auto_ctrans_eq; try (rewrite (itree_eta (x <- _;; _ x)), unfold_bind; reflexivity).
  punfold EQV. unfold_eqit.
  genobs t1 ot1.
  genobs t2 ot2.
  hinduction EQV before CIH; intros; pclearbot.
  - guclo eqit_clo_trans.
    econstructor; auto_ctrans_eq; try (rewrite <- !itree_eta; reflexivity).
    gbase; cbn.
    apply REL0; auto with itree.
  - gstep. econstructor.
    gbase.
    apply CIH.
    econstructor; eauto with itree.
  - gstep. econstructor.
    intros; apply ID; unfold id.
    gbase.
    apply CIH.
    econstructor; eauto with itree.
  - destruct b1; try discriminate.
    guclo eqit_clo_trans.
    econstructor.
    3:{ eapply IHEQV; eauto with itree. }
    3,4:auto_ctrans_eq.
    2: reflexivity.
    eapply eqit_Tau_l. rewrite unfold_bind, <-itree_eta. reflexivity.
  - destruct b2; try discriminate.
    guclo eqit_clo_trans.
    econstructor; auto_ctrans_eq; eauto with itree; try reflexivity.
    eapply eqit_Tau_l. rewrite unfold_bind, <-itree_eta. reflexivity.
Qed.

End LeafBind.

(** General cut rule for [eqit]
    This result generalizes [eqit_clo_bind].  *)
Lemma eqit_clo_bind_gen :
  forall (E) (R1 R2) (RR : R1 -> R2 -> Prop) (U1 U2) (UU : U1 -> U2 -> Prop)
         b1 b2
         (t1 : itree E U1) (t2 : itree E U2)
         (k1 : U1 -> itree E R1) (k2 : U2 -> itree E R2),
    eqit UU b1 b2 t1 t2 ->
    (forall (u1 : U1) (u2 : U2),
      u1 ∈ t1 -> u2 ∈ t2 -> UU u1 u2 ->
      eqit RR b1 b2 (k1 u1) (k2 u2)) ->
    eqit RR b1 b2 (x <- t1;; k1 x) (x <- t2;; k2 x).
Proof.
    intros.
    ginit. guclo (@eqit_Leaf_clo_bind E R1 R2).
    econstructor; eauto.
    intros * IN1 IN2 HR.
    gfinal; right.
    apply H0; auto.
Qed.
```

407 words 2,610 characters

Tower

```
Lemma eqit_clo_bind_gen :
  forall (E) (R1 R2) (RR : R1 -> R2 -> Prop) (U1 U2) (UU : U1 -> U2 -> Prop)
         b1 b2 (c : Chain (eqit_mon b1 b2))
         (t1 : itree E U1) (t2 : itree E U2)
         (k1 : U1 -> itree E R1) (k2 : U2 -> itree E R2),
    elem c _ _ UU t1 t2 ->
    (forall (u1 : U1) (u2 : U2),
      u1 ∈ t1 -> u2 ∈ t2 -> UU u1 u2 ->
      You, 2 weeks ago | 2 authors (You and one other)
      elem c _ _ RR (k1 u1) (k2 u2)) ->
    elem c _ _ RR (ITree.bind t1 k1) (ITree.bind t2 k2).
Proof.
  intros E R1 R2 RR U1 U2.
  intros UU b1 b2 c t1 t2 k1 k2.
  revert UU t1 t2 k1 k2.
  tower induction.
  intros IH.
  intros UU t1 t2 k1 k2 EQT EQKL.
  cbn [eqit_mon body] in *.
  unfold eqit_ in *.
  genobs t1 ot1.
  genobs t2 ot2.
  hinduction EQT before RR; intros.
  1-3: rewrite 2 observe_bind; simpobs.
  + apply EQKL.
    * apply LeafRet; auto.
    * apply LeafRet; auto.
    * exact REL.
  + taus.
    eapply IH.
    * exact REL.
    * intros u1 u2 HL1 HL2 HU.
      step. apply EQKL.
      -- eapply LeafTau; eauto.
      -- eapply LeafTau; eauto.
      -- exact HU.
  + constructor. intro v.
    eapply IH.
    * apply REL.
    * intros u1 u2 HL1 HL2 HU.
      step. apply EQKL.
      -- eapply LeafVis; eauto.
      -- eapply LeafVis; eauto.
      -- exact HU.
  + rewrite observe_bind. simpobs.
    taul.
    eapply IHEQT; eauto with itree.
  + setoid_rewrite observe_bind at 2. simpobs.
    taur.
    eapply IHEQT; eauto with itree.
Qed.
```

258 words 1,364 characters

Up-to proofs got shorter

Broadly, about a 40-50% decrease in proof and definition verbosity

Best recorded improvement:

1,490 words 9,725 characters

570 words 4,391 characters

Some observations, pt. 2

Paco

```
Lemma eutt_conj {E} {R S} {RS RS'} :
  forall (t : itree E R) (s : itree E S),
    eutt RS  t s ->
    eutt RS' t s ->
    eutt (RS /2\ RS') t s.
Proof.
  repeat red.
  einit. ecofix CIH. intros * EQ EQ'.
  rewrite itree_eta, (itree_eta s).
  punfold EQ; punfold EQ'; red in EQ; red in EQ'.
  genobs t ot; genobs s os.
  hinduction EQ before CIHH; subst; intros; pclearbot; simpl.

  - estep; split; auto.
    inv EQ'; auto.
  - estep; ebase; right; eapply CIHL; eauto.
    rewrite <- tau_eutt.
    rewrite <- (tau_eutt m2); auto with itree.
  - assert (EE := eqitF_inv_VisF _ _ _ _ _ EQ'); pclearbot.
    eapply euttG_vis; ebase; left; apply CIHH; auto with itree.
  - eapply fold_eqitF in EQ'; eauto.
    assert (t ≈ Tau t1) by (rewrite itree_eta, <- Heqot; reflexivity).
    rewrite H in EQ'.
    apply eqit_inv_Tau_l in EQ'.
    subst; specialize (IHEQ _ _ eq_refl eq_refl).
    punfold EQ'; red in EQ'.
    specialize (IHEQ EQ').
    rewrite eqit_Tau_l; [|reflexivity].
    rewrite (itree_eta t1).
    eapply IHEQ.
  - subst; cbn.
    rewrite tau_euttge.
    rewrite (itree_eta t2); eapply IHEQ; eauto.
    eapply fold_eqitF in EQ'; eauto.
    assert (s ≈ Tau t2).
    rewrite (itree_eta s), <- Heqos; reflexivity.
    rewrite tau_eutt in H.
    assert (eutt RS' t t2).
    rewrite <- H; auto.
    punfold H0.
Qed.
```

209 words 1,281 characters

Proofs by coinduction got shorter

They also use fewer library tactics:

Tower

```
Lemma eutt_conj {E} {R S} {RS RS'} :
  forall (t : itree E R) (s : itree E S),
    eutt RS  t s ->
    eutt RS' t s ->
    eutt (conj_rel RS RS') t s.
Proof.
  icoinduction c CIH. intros * EQ EQ'.
  step in EQ; step in EQ'.
  genobs t ot; genobs s os.
  hinduction EQ before CIH; subst; intros; simpl.
  - inv EQ'. eret. now constructor.
  - taus. eapply CIH; eauto. apply eqit_inv_Tau. now step.
  - constructor. intro v. specialize (REL v).
    eapply CIH; eauto.
    now eapply eqitF_inv_VisF in EQ'; eauto.
  - taul. eapply IHEQ; eauto. subst. unstep. eapply eqit_inv_Tau_l.
    now step.
  - taur. eapply IHEQ; eauto. subst. unstep. eapply eqit_inv_Tau_r.
    now step.
Qed.
```

117 words 671 characters

This example: 7 vs 2

Other informal observations

Proofs by coinduction and up-to proofs look very similar

We depend on a (much) smaller library

Proofs are easier for me to read and explain

LOC # is theoretically* much lower

No need to remember arities, everything is `gfp`

Ideas and Future Work

Main Ambition: even better up-to with diacritical companions.

Imagine a proof by coinduction of *weak* bisimilarity (bisimilarity modulo invisible steps).

$$\tau \quad \tau \quad \tau \quad \beta \quad \tau \quad \tau \quad \tau \quad \ldots \quad \approx \quad \beta \quad \ldots$$

We can rewrite by **strong bisimilarity**, but **not by weak bisimilarity** in general.

There are special cases where we can, though!

How? Only if we use CIH under beta, not tau.

`gpaco` lets us do this, and so does the *diacritical companion*, a newer discovery.

Can we recover this power in the land of the tower?

Ideas and Future Work

Main Ambition: even better up-to with diacritical companions

There exists modern work which could recover some of the power of gpaco within the tower framework, but if this is usable remains to be seen (part of my summer work at Inria)

Ideas and Future Work

1. Main Ambition: even better up-to with diacritical companions

There exists modern work which could recover some of the power of gpaco within the tower framework, but if this is usable remains to be seen (part of my summer work at Inria)

2. A write-up on the tower in the library

Because history is nonlinear, no write-up of the enhanced coinduction library as it is exists

3. Damien Pous pls accept my pr

Thank you!

Usability of the enhanced coinduction library is good, but there are bugs

Also, there are typical design patterns with the library that probably should be part of it.

Appendix

Review: F-Algebras and Coalgebras

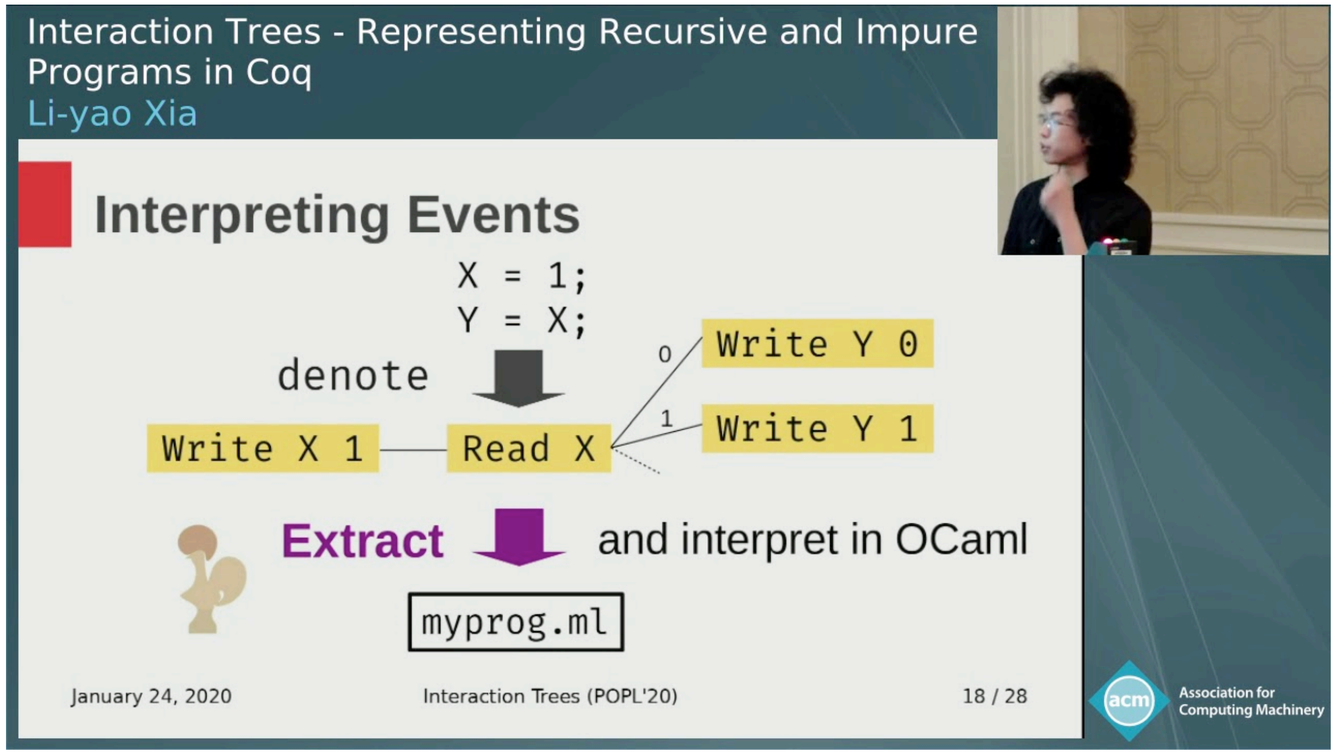
- F-algebra: A set A and a map α s.t. α `:` $F\ A$ `->` A maps operations with arguments from A to A

e.g. `FX = 1 + X × X` on ***Set*** means α is a map from

`1 + A × A` to `A`

- Concretely, this is how we get Inductives : initially guarantees members of an inductive type are only built from its constructors.

Paco -> ITrees

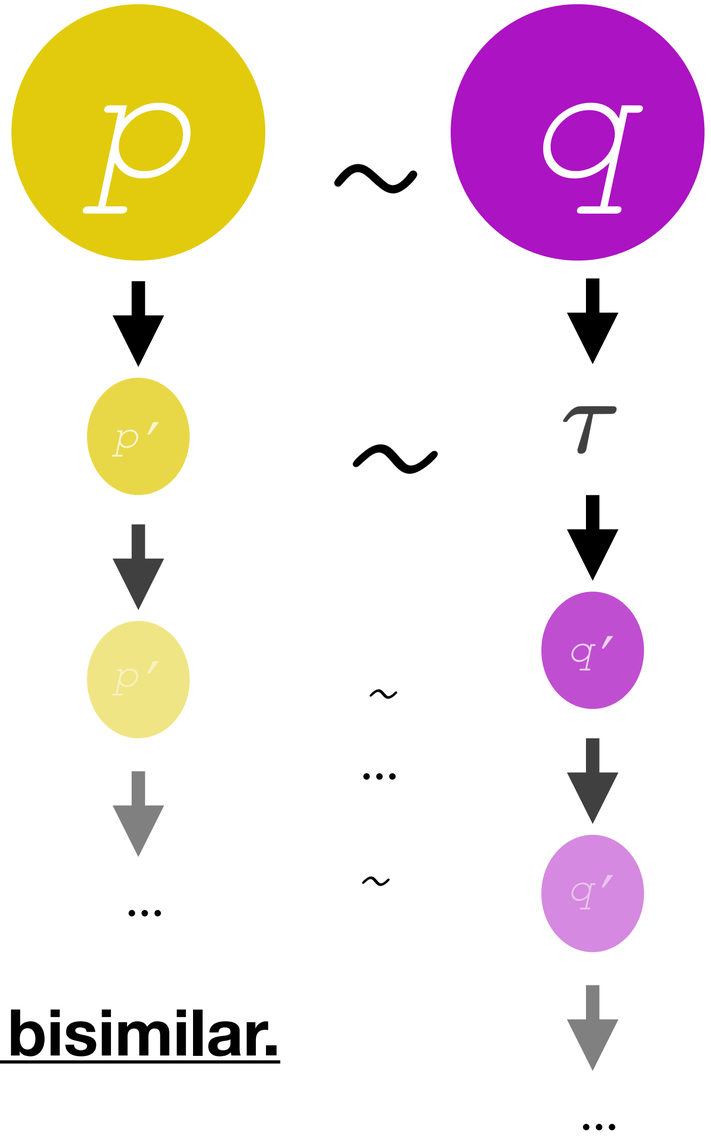

Interaction Trees - Representing Recursive and Impure Programs in Coq
Li-yao Xia
Interpreting Events
X = 1;
Y = X;
denote
Write X 1
Read X
0
1
Write Y 0
Write Y 1
Extract
and interpret in OCaml
myprog.ml
January 24, 2020
Interaction Trees (POPL'20)
18 / 28
Association for Computing Machinery

Notice Tau: ‘invisible steps’

```
(** The type [itree] is defined as the final coalgebra ("greatest
    fixed point") of the functor [itreeF]. *)
Variant itreeF (itree : Type) :=
| RetF (r : R)
| TauF (t : itree)
| VisF {X : Type} (e : E X) (k : X -> itree)
.

(** We define non-recursive types such as [itreeF] using the [Variant]
    command. The main practical difference from [Inductive] is that
    [Variant] does not generate any induction schemes (which are
    unnecessary). *)

CoInductive itree : Type := go
{ _observe : itreeF itree }.
```



If p, q are equivalent “up to τ ”, they are <u>weakly bisimilar.</u>